

虚拟内存

古金宇

上海交通大学并行与分布式系统研究所

<https://ipads.se.sjtu.edu.cn>

Introduction

- **Memory is critical for AI Agents**
 - MemGPT: Towards LLMs as Operating Systems (ICLR 2024)
 - A-MEM: Agentic Memory for LLM Agents (NeurIPS 2025)
 - Memory OS of AI Agent (EMNLP 2025)
 - Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory (arXiv 2025)
- **Let us understand how OS manage memory first!**

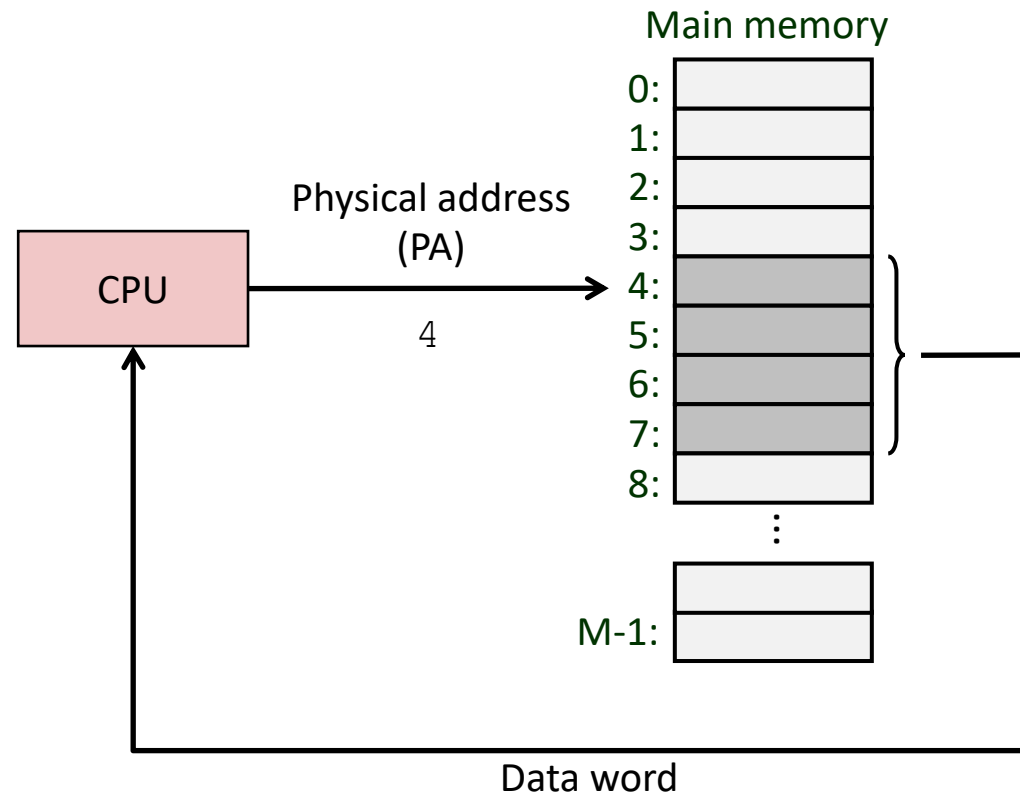
Outline

- **Physical and Virtual Addressing**
- **Address Spaces**
- **VM as a Tool for Caching**
- **VM as a Tool for Memory Management**
- **VM as a Tool for Memory Protection**
- **Suggested reading: CSAPP 9.1~9.5**

Physical Addressing

- **Attributes of the main memory**
 - Organized as an array of M contiguous byte-sized cells
 - Each byte has a unique **physical address** (PA) started from 0
- **physical addressing**
 - A CPU use physical addresses to access memory
- **Examples**
 - Early PCs, DSP, embedded microcontrollers

A System Using Physical Addressing



- Used in “simple” systems like embedded microcontrollers

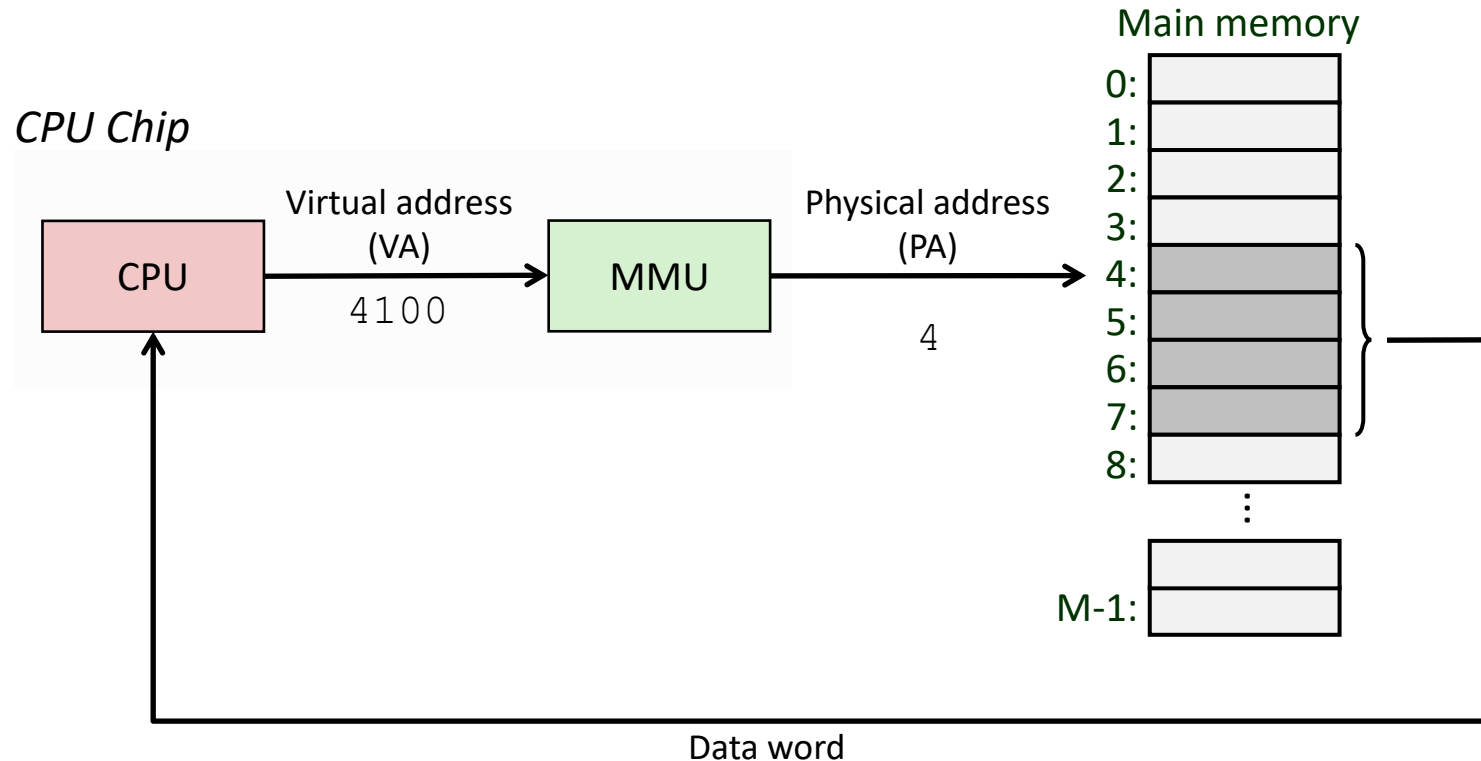
Virtual Addressing

- **Virtual addressing**
 - CPU accesses main memory by a **virtual address** (VA)
 - The virtual address is converted to the appropriate physical address

Virtual Addressing

- **Address translation**
 - Converting a **virtual address** to a **physical address**
 - Requires close **cooperation** between the CPU hardware and the operating system
 - HW: the memory management unit (MMU)
 - Dedicated hardware on the CPU chip to translate virtual addresses on the fly
 - SW: A look-up table
 - Stored in main memory
 - Contents are managed by the operating system

A System Using Virtual Addressing



- Used in all modern servers, desktops, and laptops
- One of the great ideas in computer science

Address Space

- **Address Space**
 - An ordered set of nonnegative integer addresses
- **Linear Space**
 - The integers in the address space are consecutive
- **Linear address space:** Ordered set of contiguous non-negative integer addresses:
 $\{0, 1, 2, 3 \dots\}$

Address Space

- **N-bit Address Space**
 - **Virtual address space:**
Set of $N = 2^n$ virtual addresses $\{0, 1, 2, 3, \dots, N-1\}$
 - **Physical address space:**
Set of $M = 2^m$ physical addresses $\{0, 1, 2, 3, \dots, M-1\}$
- **Each object can now have multiple addresses**
 - one physical address, one (or more) virtual addresses

Address Space

- $K=2^{10}$ (Kilo), $M=2^{20}$ (Mega), $G=2^{30}$ (Giga), $T=2^{40}$ (Tera),
 $P=2^{50}$ (Peta), $E=2^{60}$ (Exa)

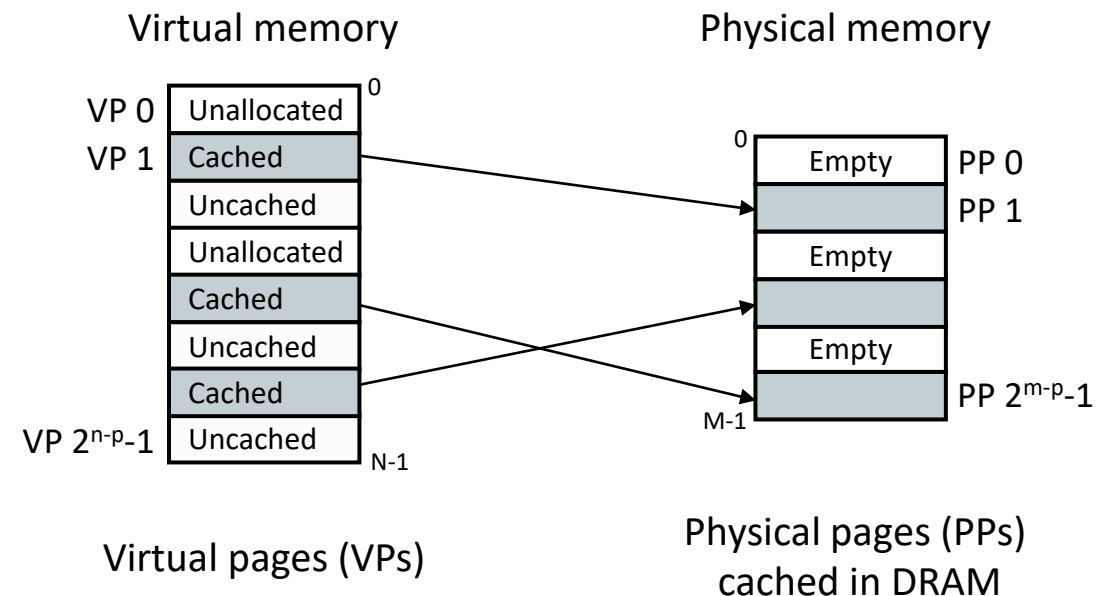
#virtual address bits (n)	#virtual address (N)	Largest possible virtual address
8	256	255
16	64K	64K-1
32	4G	4G-1
48	256T	256T-1
64	16E	16E-1

Why Virtual Memory (VM)?

- **Uses main memory efficiently**
 - Use DRAM as a cache for the parts of a virtual address space
- **Simplifies memory management**
 - Each process gets the same uniform linear address space
- **Isolates address spaces**
 - One process can't interfere with another's memory
 - User program cannot access privileged kernel information

Page

- Each page is 2^p bytes (usually, 4K) in size
 - Serve as the transfer units between the disk and the main memory
 - virtual pages (VPs)
 - physical pages (PPs)
 - or page frames



Q: Where are the uncached pages?

Page Attributes

- **Unallocated:**

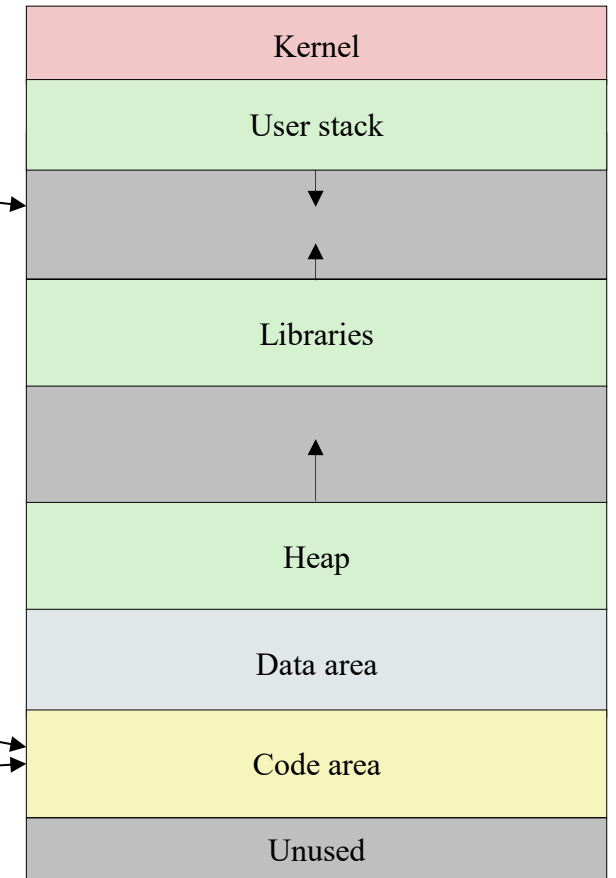
- Pages that have not yet been allocated (or created) by the VM system
- Do not have any data associated with them

- **Cached:**

- Allocated pages that are currently cached in physical memory

- **Uncached:**

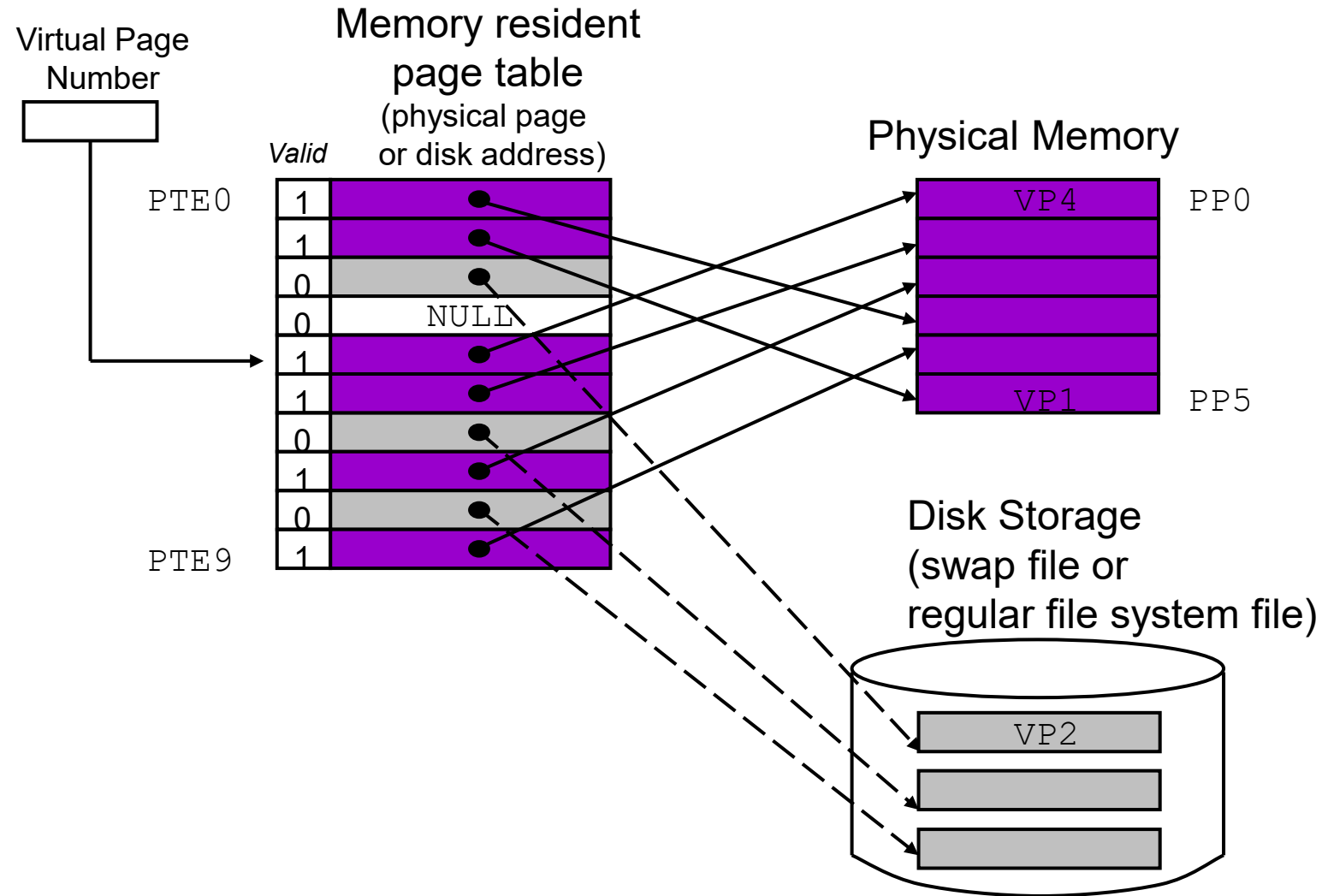
- Allocated pages that are not cached in physical memory



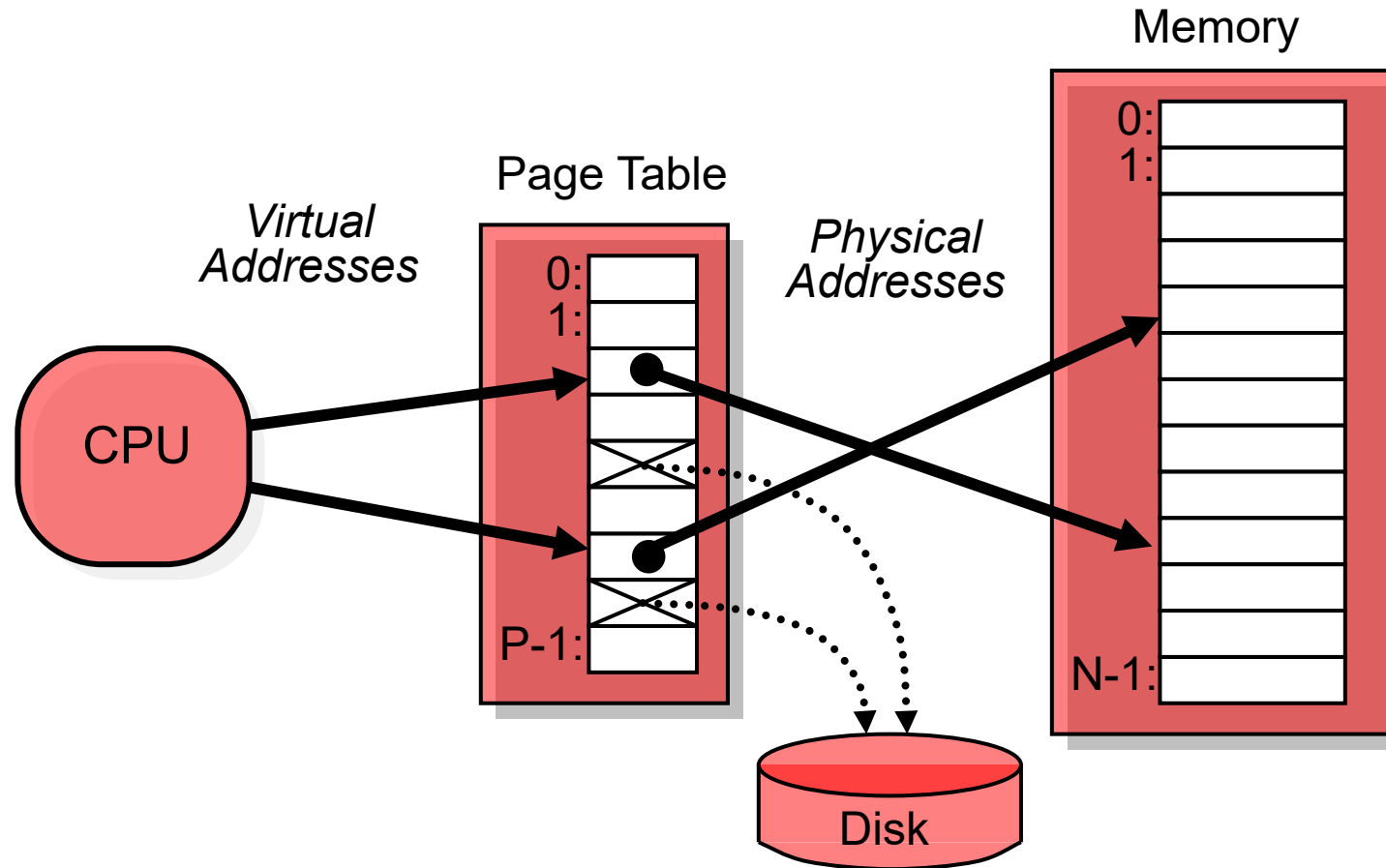
Page Table

- **Mapping** from virtual pages to physical pages
 - From uncached form to cached form
- **Each allocated page of virtual memory has entry in (single-level) page table**
 - if page is in memory
 - page table entry specifies physical address
 - if page is not in memory
 - page table entry specifies disk address
 - OS retrieves information

Page Table



Page Hits



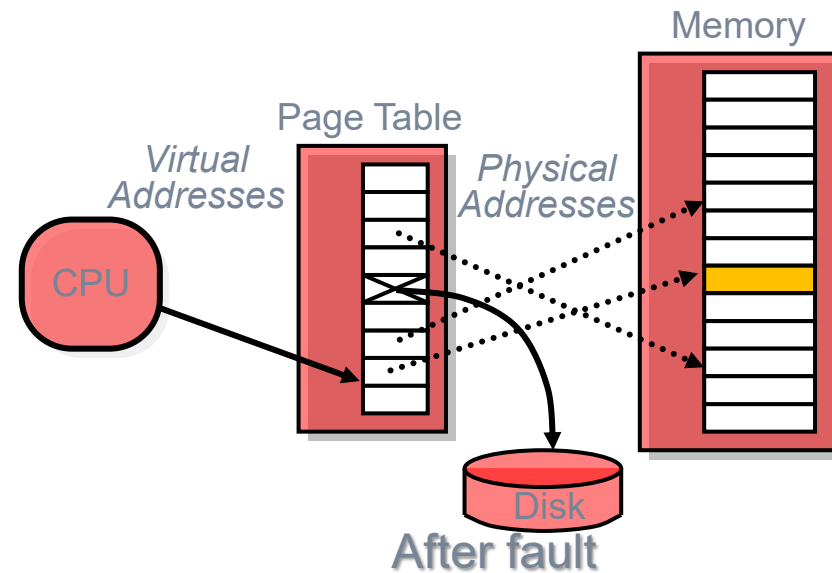
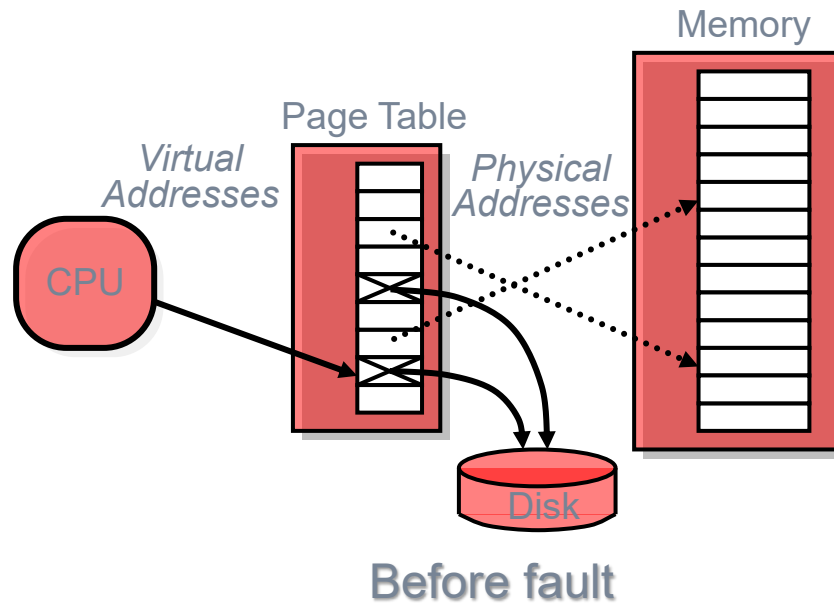
Address Translation: Hardware converts *virtual addresses* to *physical addresses* via a page table managed by OS

Page Faults

- Page table entry indicates virtual address **not in memory**
 - Three possible cases
- **Case-1: OS exception handler invoked to move data from disk into memory**
 - current process suspends, others can resume
 - OS has full control over placement, etc.

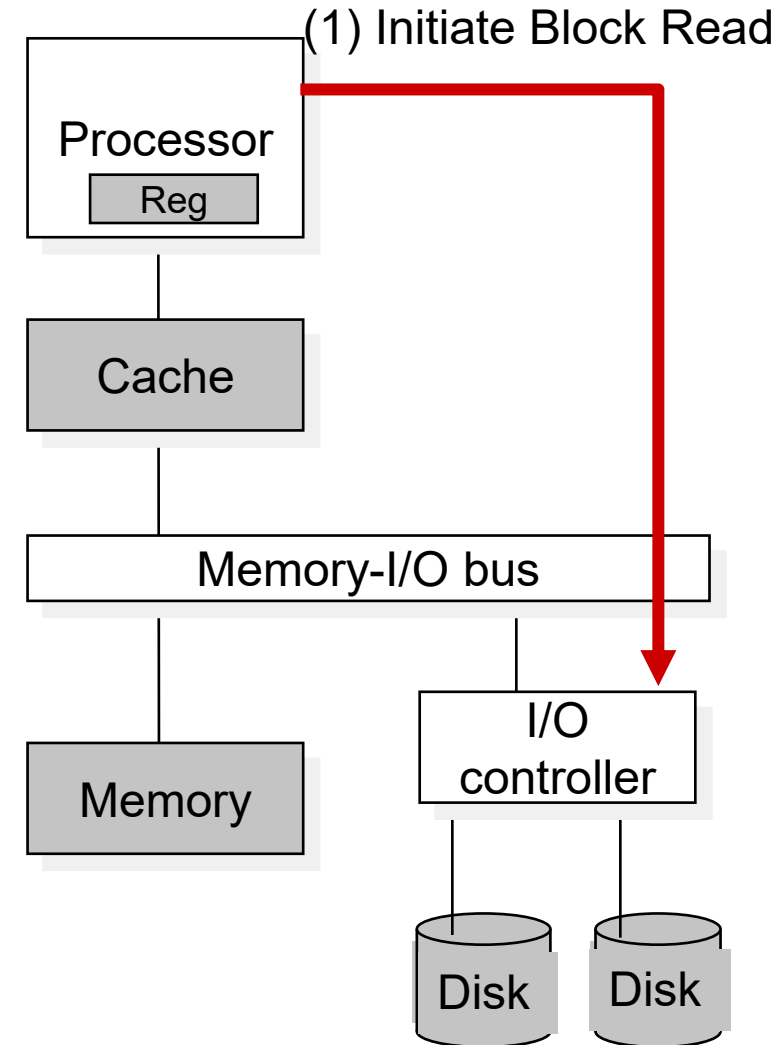
Page Faults

- **Case-1: Swapped in or paged in**
 - Swapped out or paged out
- **Case-2: Demand paging**
 - File (swapped in) or Anonymous pages (e.g., heap)
- **Case-3: Segmentation fault**



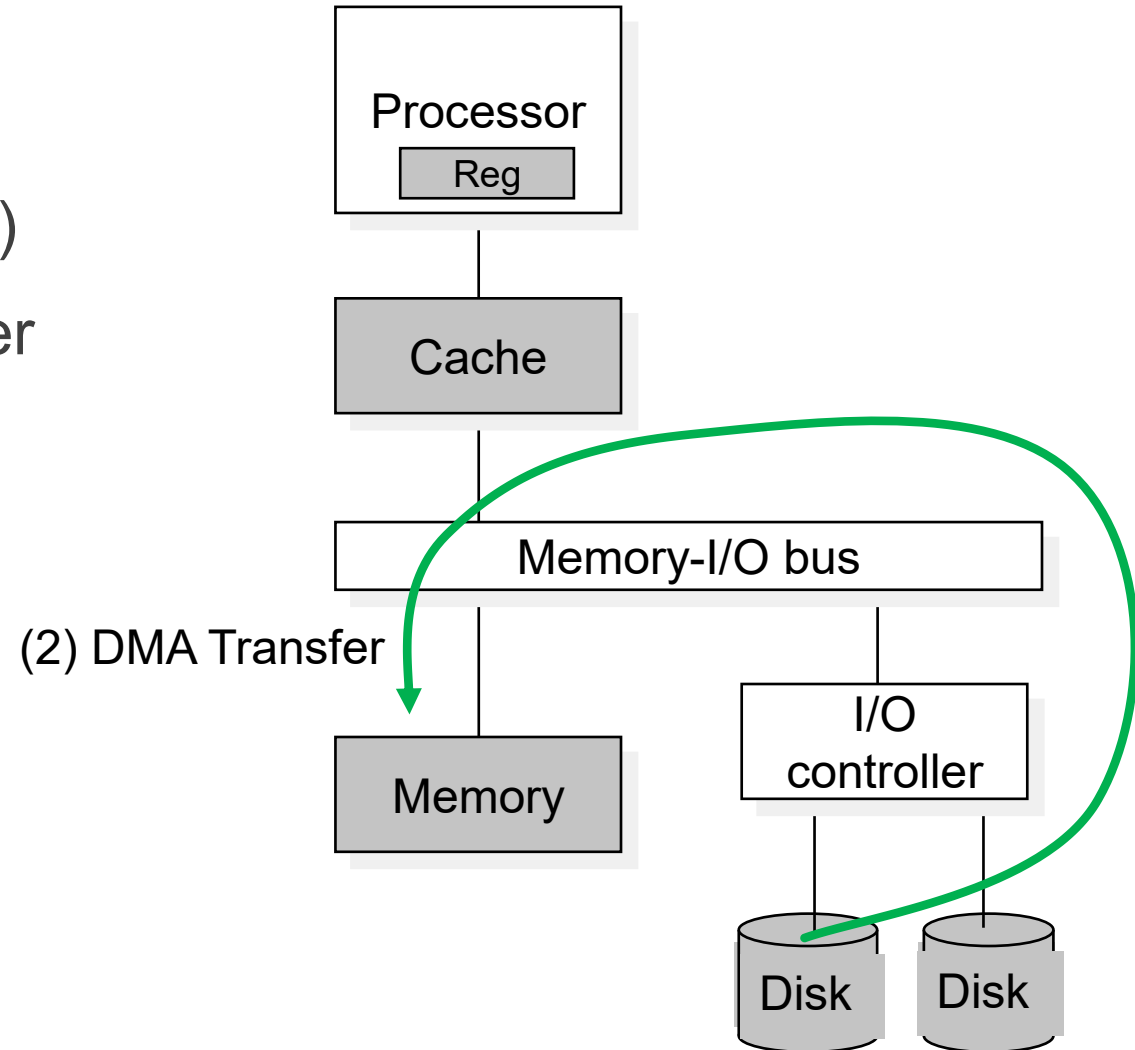
Servicing a Page Fault

- **Processor Signals Controller**
 - Read block of length P starting at disk address X and store starting at memory address Y



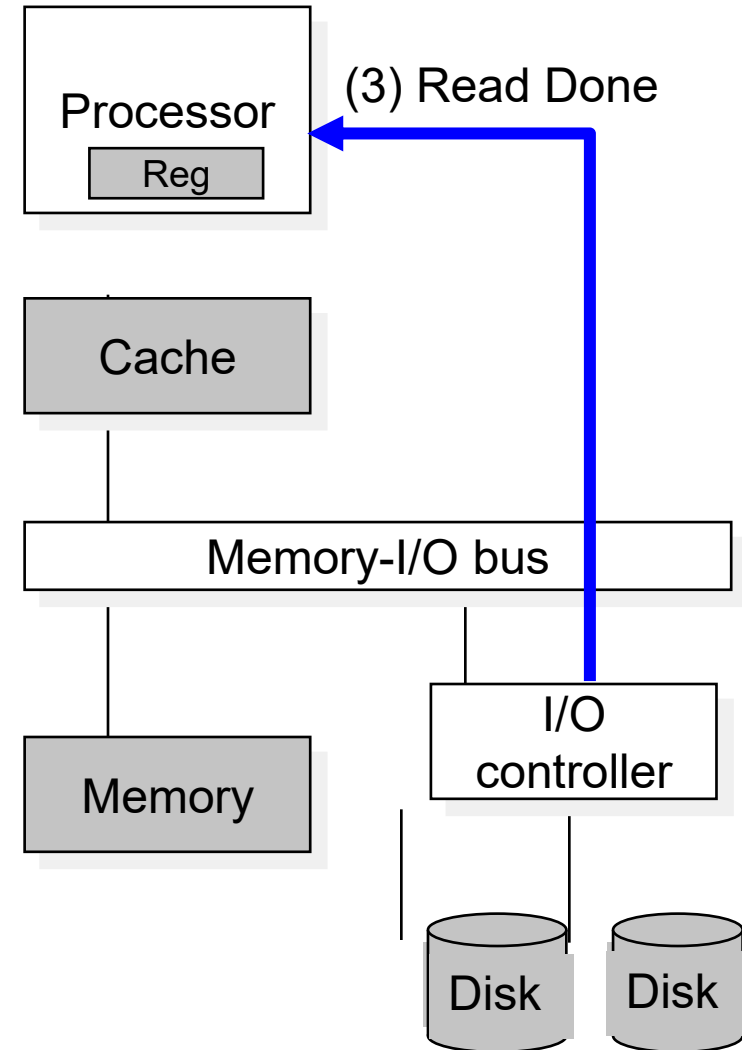
Servicing a Page Fault

- **Read Occurs**
 - Direct Memory Access (DMA)
 - Under control of I/O controller



▶ Servicing a Page Fault

- **I / O Controller Signals Completion**
 - Interrupt processor
 - OS resumes suspended process



Locality to the Rescue Again!

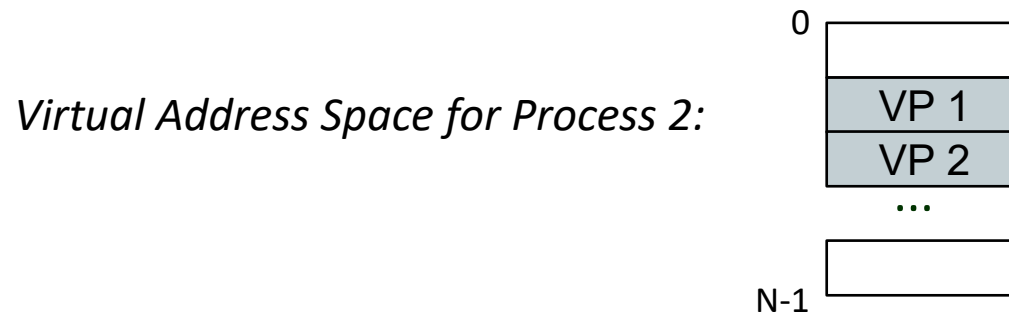
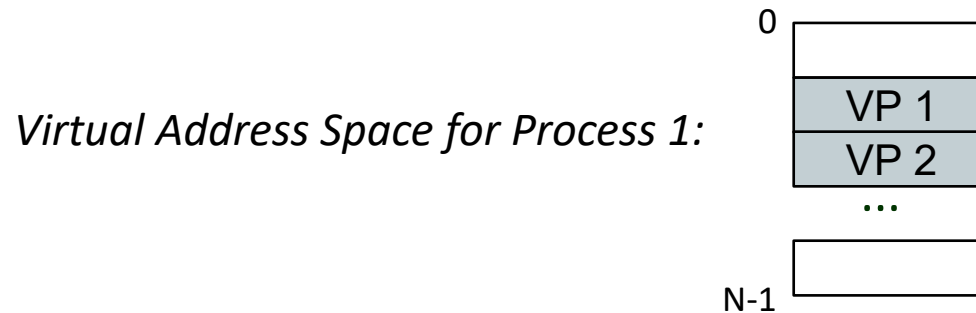
- Virtual memory works because of locality
- At any point in time, programs tend to access a set of active virtual pages called the **working set**
- Programs with better **temporal** locality will have smaller working sets
- If (working set size $<$ main memory size)
 - Good performance for one process after compulsory misses
- If (working set sizes $>$ main memory size)
 - **Thrashing**: Performance meltdown where pages are swapped (copied) in and out continuously

Why Virtual Memory (VM)?

- **Uses main memory efficiently**
 - Use DRAM as a cache for the parts of a virtual address space
- **Simplifies memory management**
 - Each process gets the same uniform linear address space
- **Isolates address spaces**
 - One process can't interfere with another's memory
 - User program cannot access privileged kernel information

VM as a Tool for Memory Management

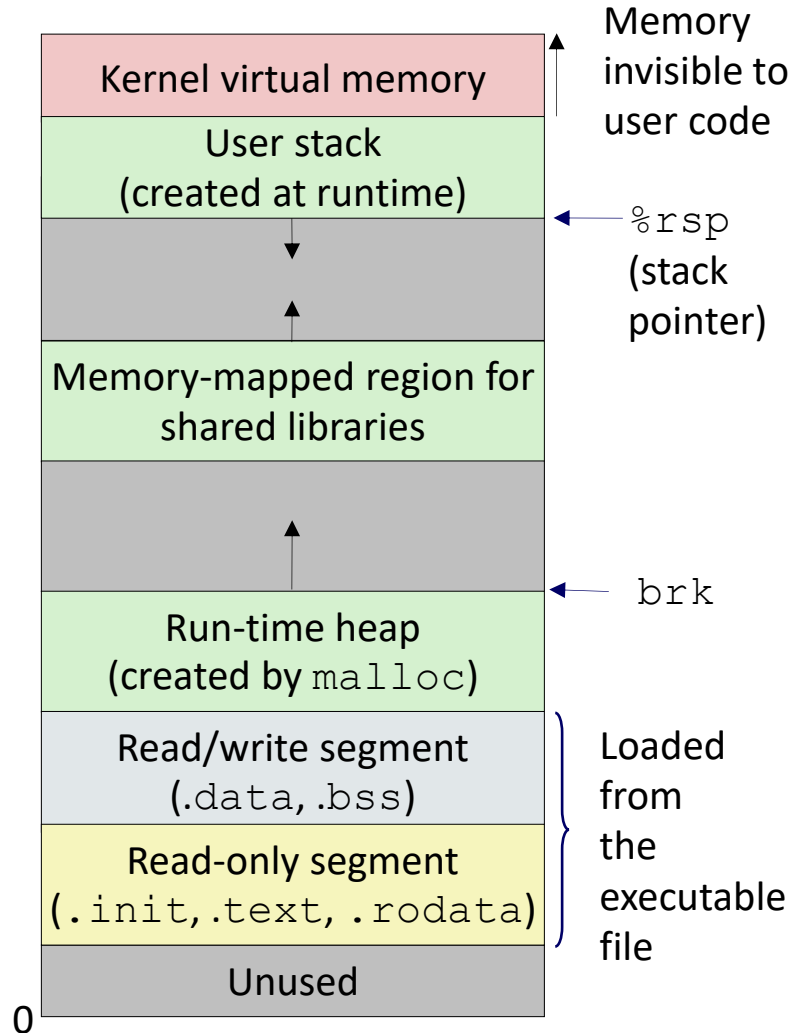
- Key idea: each process has its **own** virtual address space
 - It can view memory as a simple linear array



Simplifying Linking and Loading

- **Linking**

- Each program has **similar** virtual address space
- Code, stack, and shared libraries always start at the **same** address

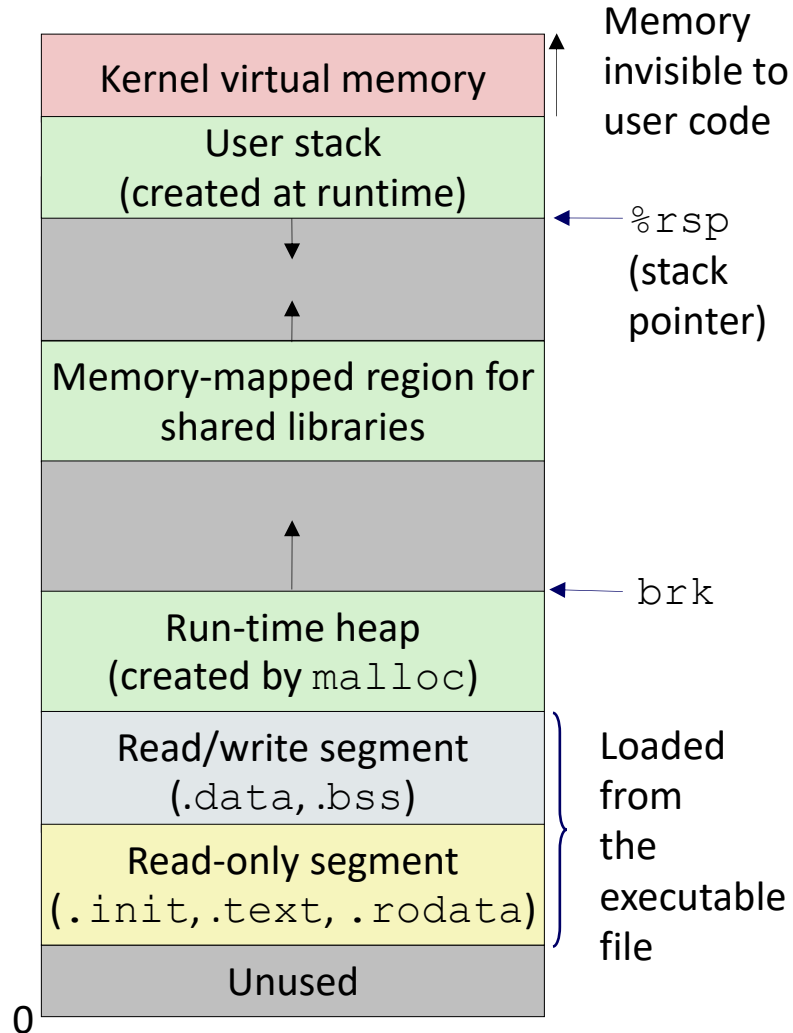


Simplifying Linking and Loading

- **Loading**

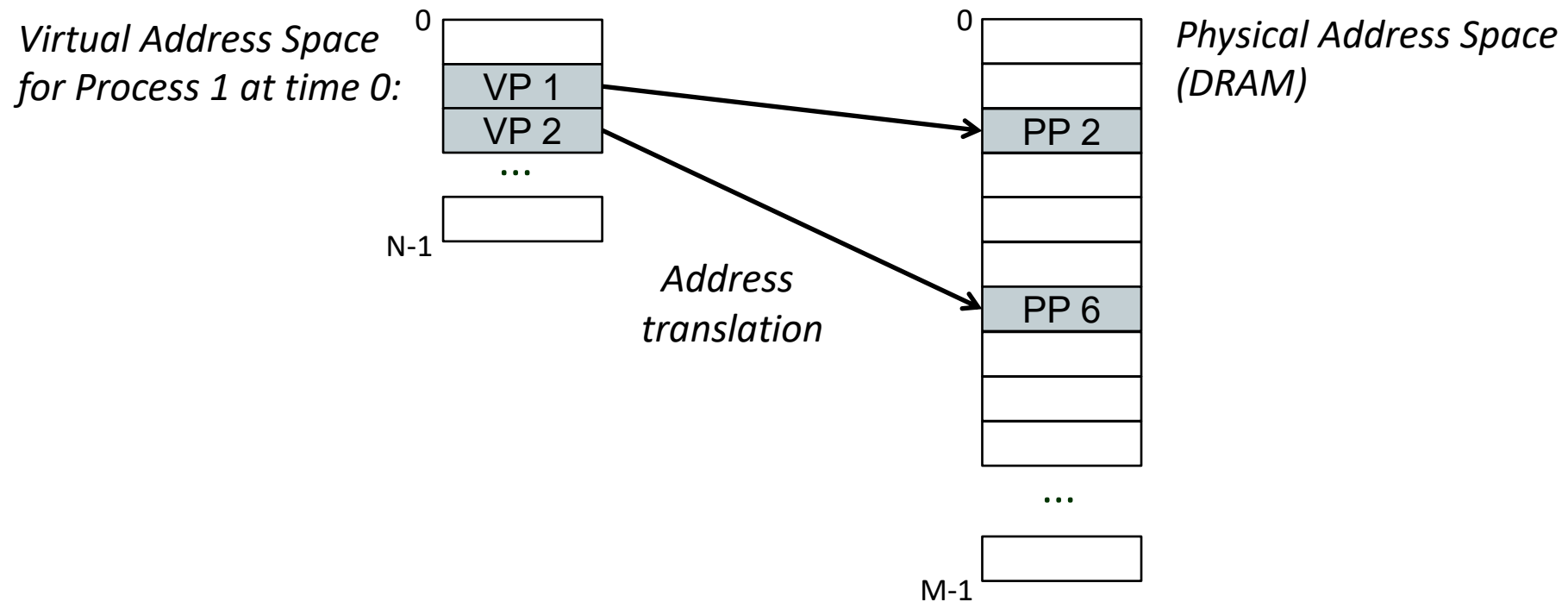
- **execve()** allocates virtual pages for `.text` and `.data` sections
= creates PTEs marked as **invalid**
- The `.text` and `.data` sections are copied, page by page, **on demand** by the virtual memory system

Q: What is the benefit of on-demand paging for `execve`?



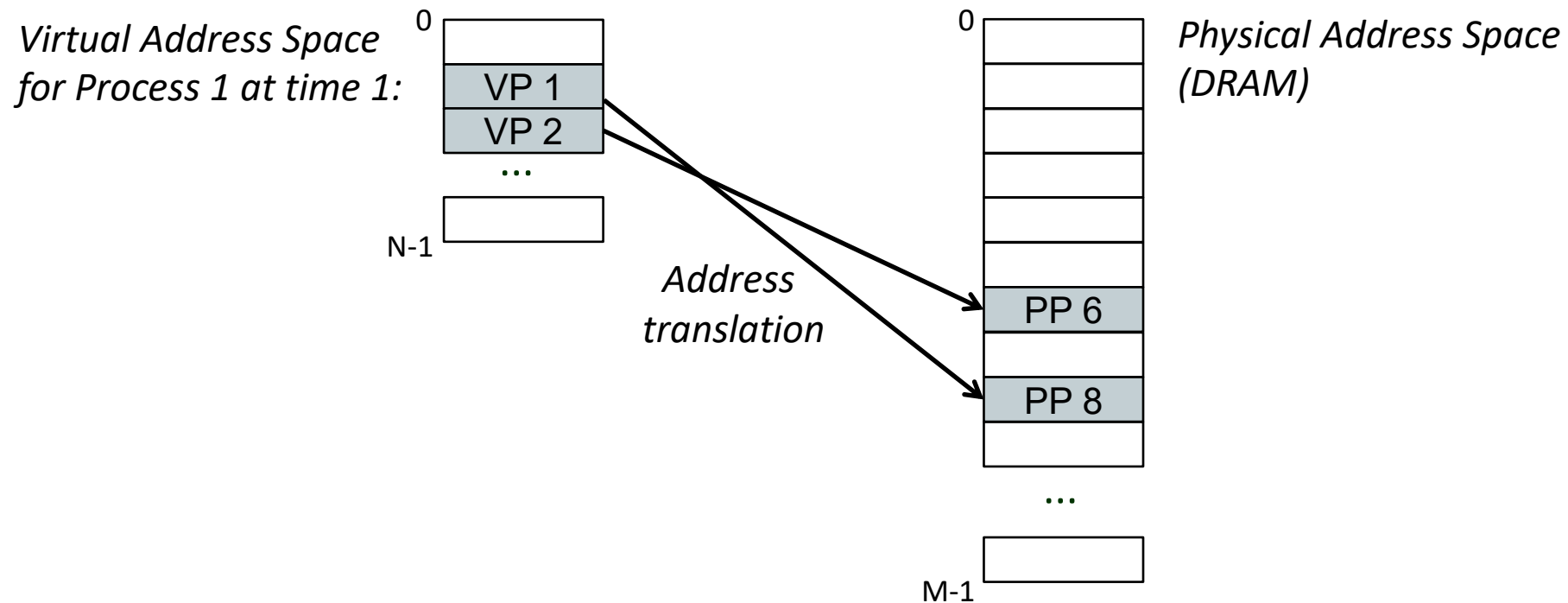
VM as a Tool for Memory Management

- **Memory allocation**
 - Each virtual page can be mapped to any physical page
 - A virtual page can be stored in different physical pages **at different times**



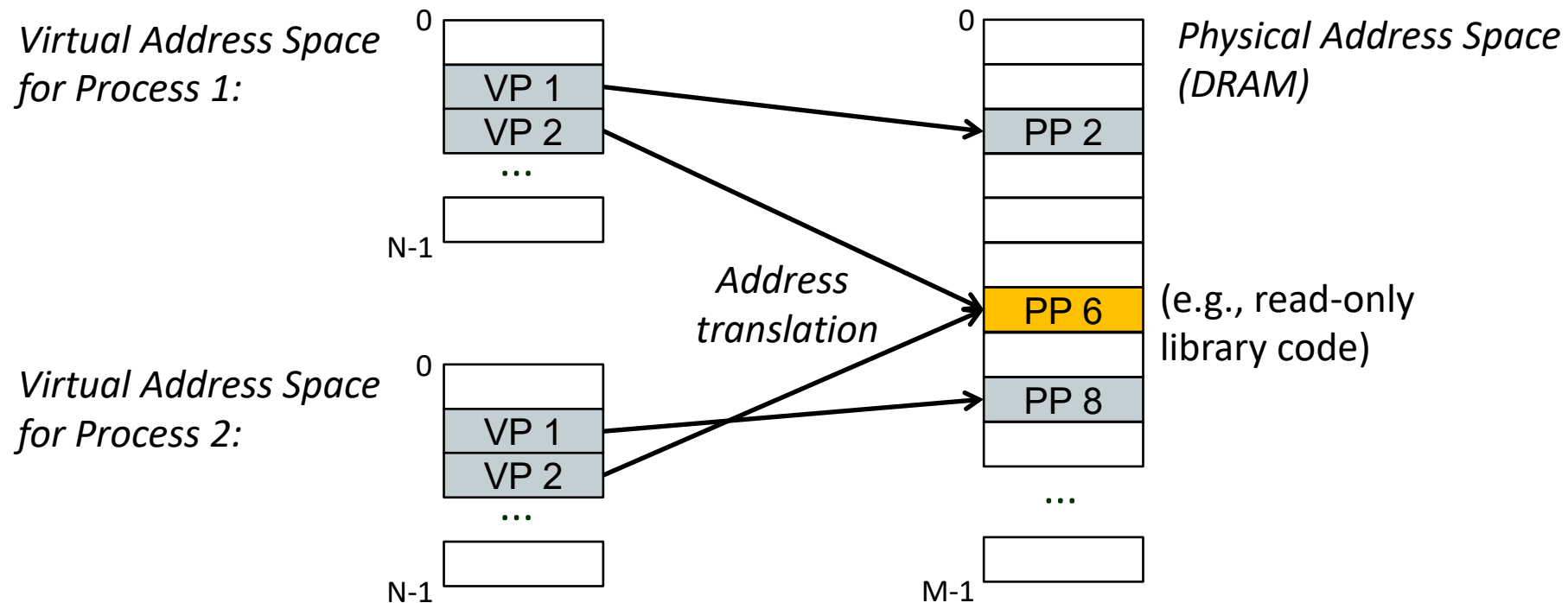
VM as a Tool for Memory Management

- **Memory allocation**
 - Each virtual page can be mapped to any physical page
 - A virtual page can be stored in different physical pages **at different times**



VM as a Tool for Memory Management

- **Sharing** code and data among processes
 - Map virtual pages to the same physical page (e.g., PP 6)

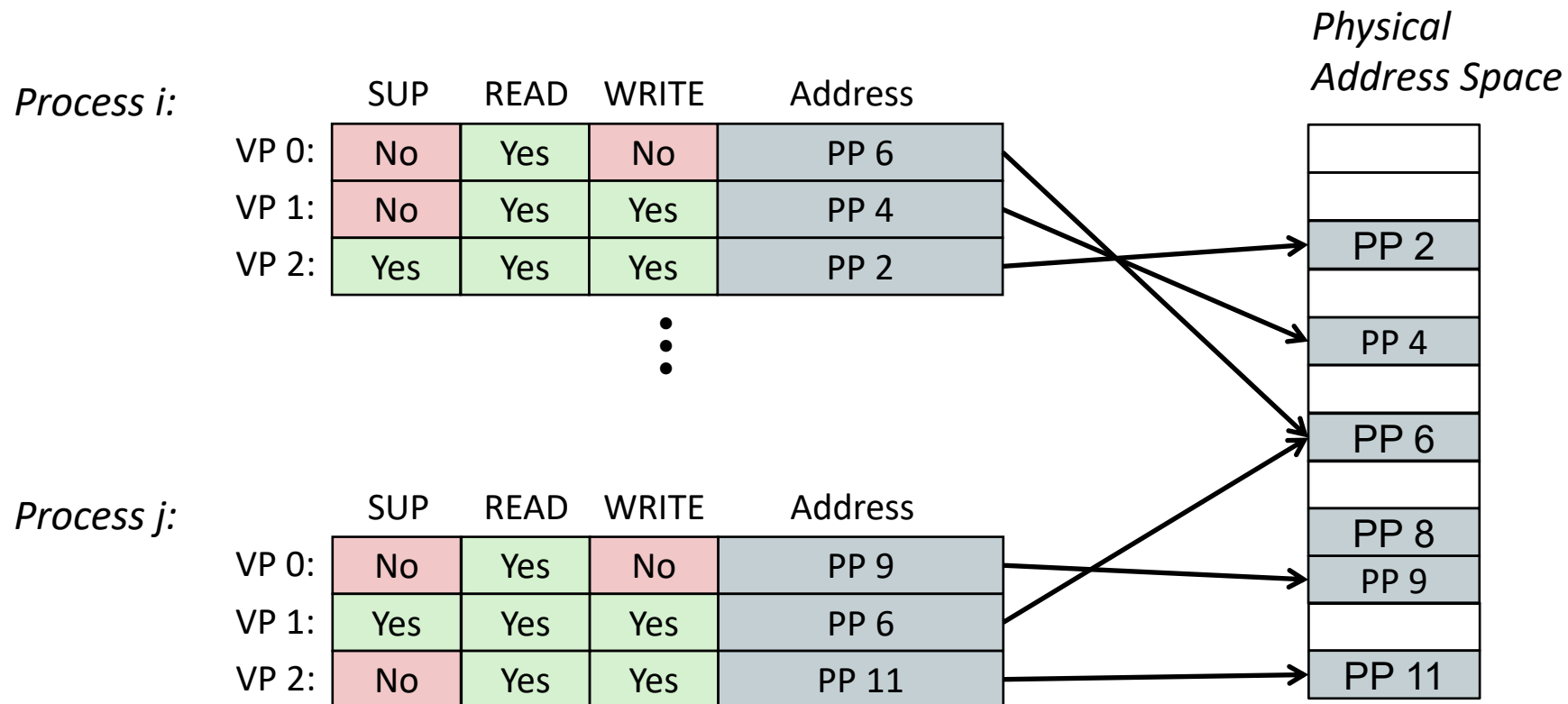


Why Virtual Memory (VM)?

- **Uses main memory efficiently**
 - Use DRAM as a cache for the parts of a virtual address space
- **Simplifies memory management**
 - Each process gets the same uniform linear address space
- **Isolates address spaces**
 - One process can't interfere with another's memory
 - User program cannot access privileged kernel information

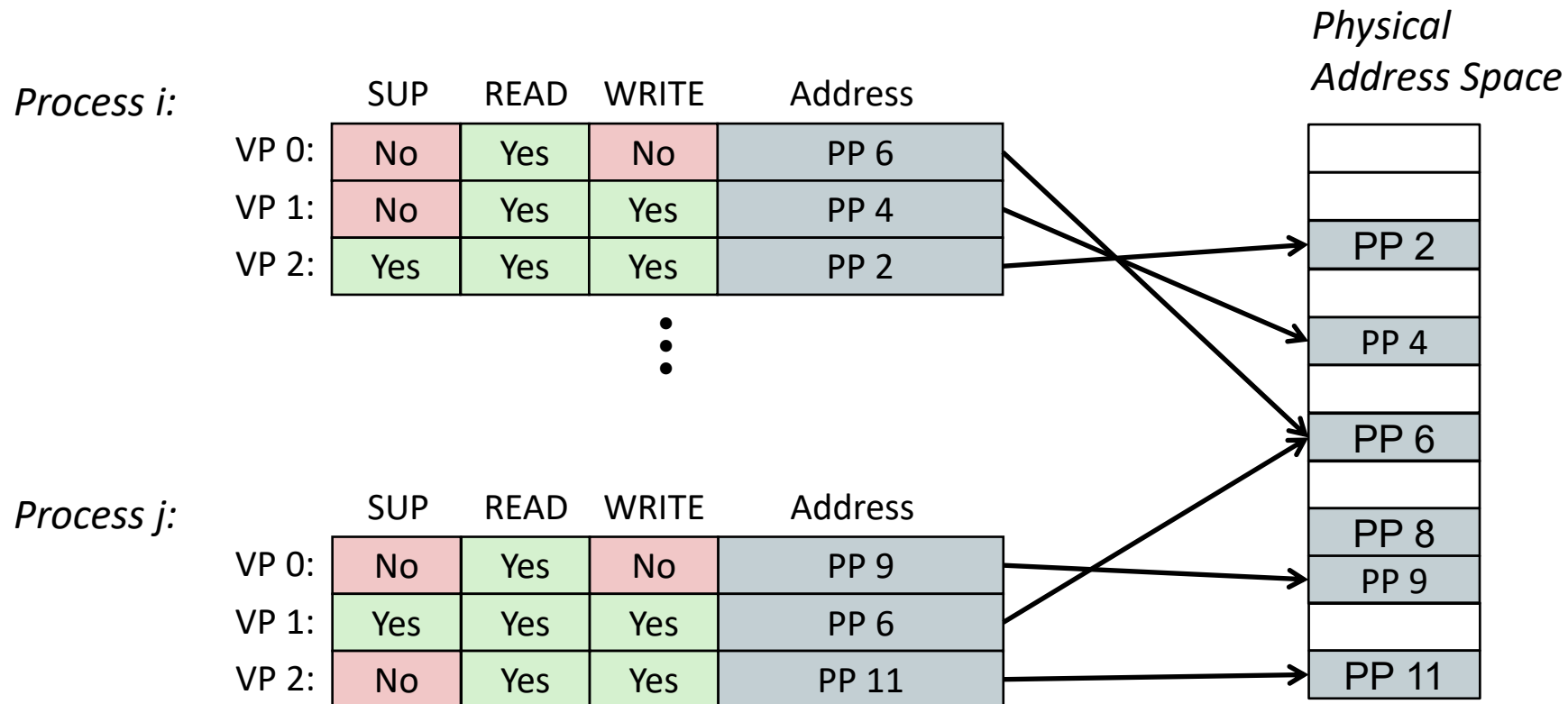
VM as a Tool for Memory Protection

- **Extend PTEs with permission bits**
 - The same physical page has different permission for different process



VM as a Tool for Memory Protection

- **Page fault handler checks these before remapping**
 - If violated, send process SIGSEGV (segmentation fault)



Memory Address Translation

地址翻译

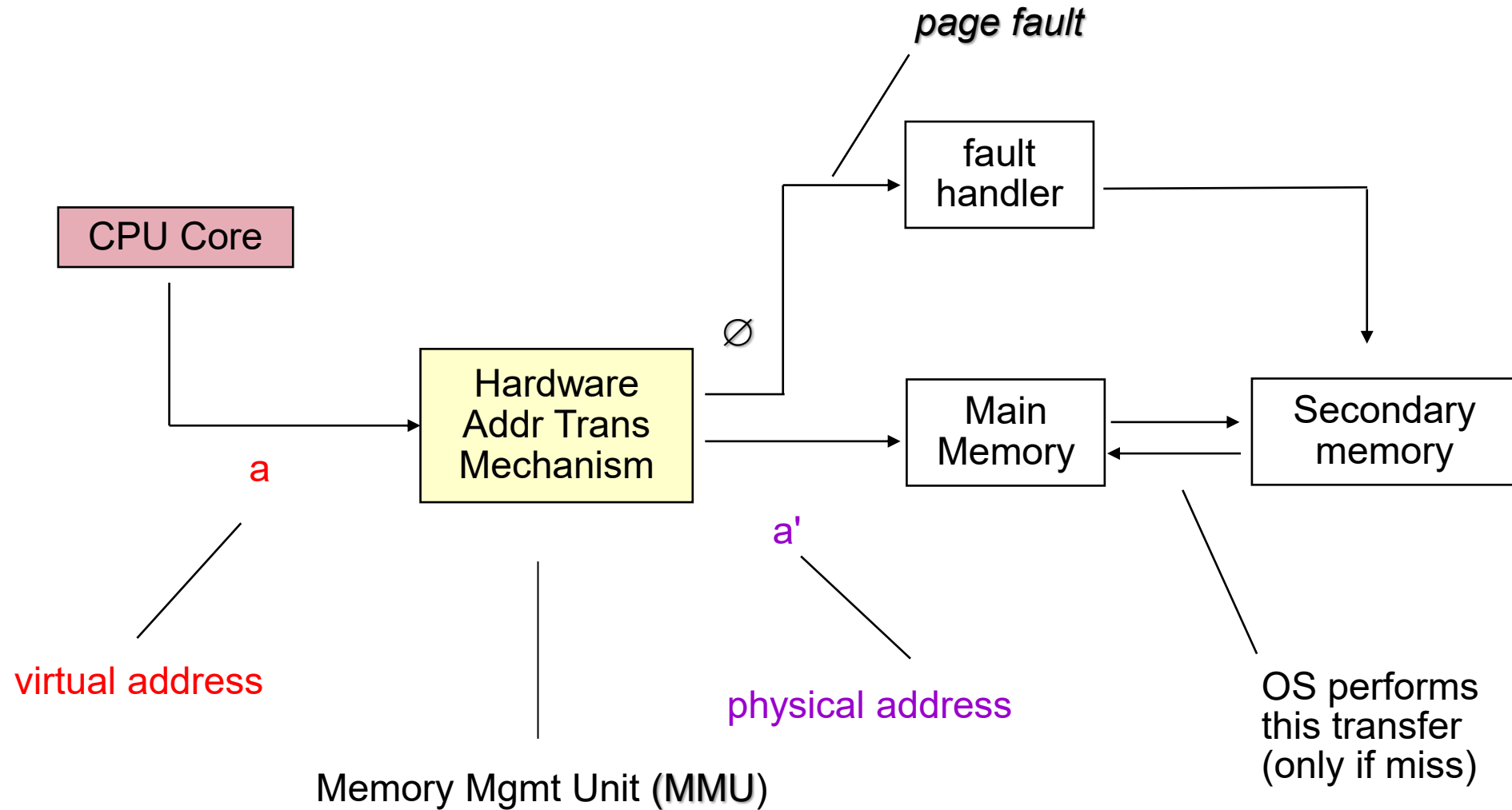
Outline

- **Address translation**
- **Accelerating translation**
 - with a TLB
 - Multilevel page tables
- **An Intel x86-64 CPU Memory System**
- **Suggested reading: 9.6, 9.7.1**

Address Translation

- $V = \{0, 1, \dots, N-1\}$ **virtual** address space
- $P = \{0, 1, \dots, M-1\}$ **physical** address space
- Address Translation
- MAP: $V \rightarrow P \cup \{\emptyset\}$ address mapping function
- MAP(**a**) = **a'** if data at virtual address **a** is
present at physical address **a'** in P
= $\emptyset$ if data at virtual address **a** is not
present in P (invalid or on disk)

Address Translation

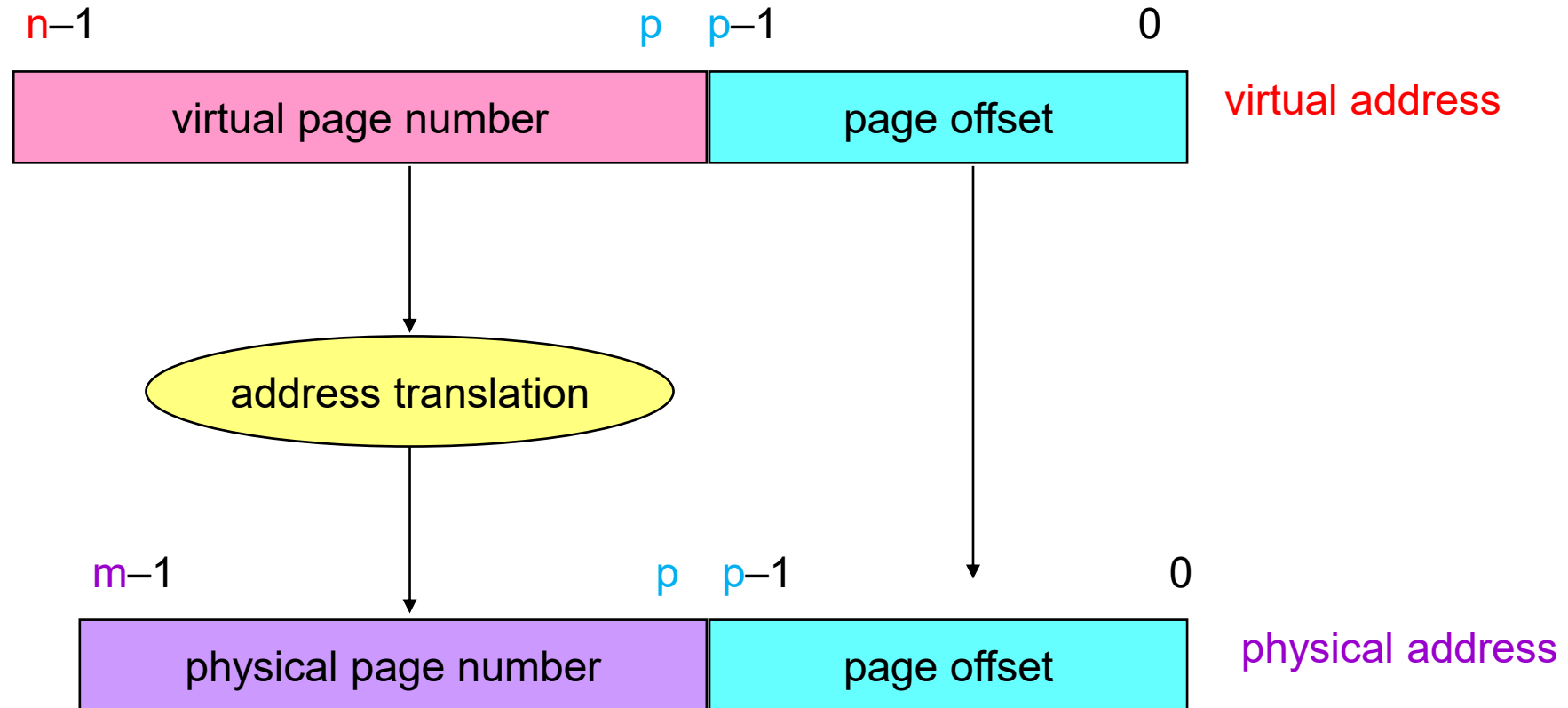


Address Translation

- **Basic Parameters**

- $N = 2^n$ = Virtual address limit
- $M = 2^m$ = Physical address limit
- $P = 2^p$ = page size (bytes)

Address Translation

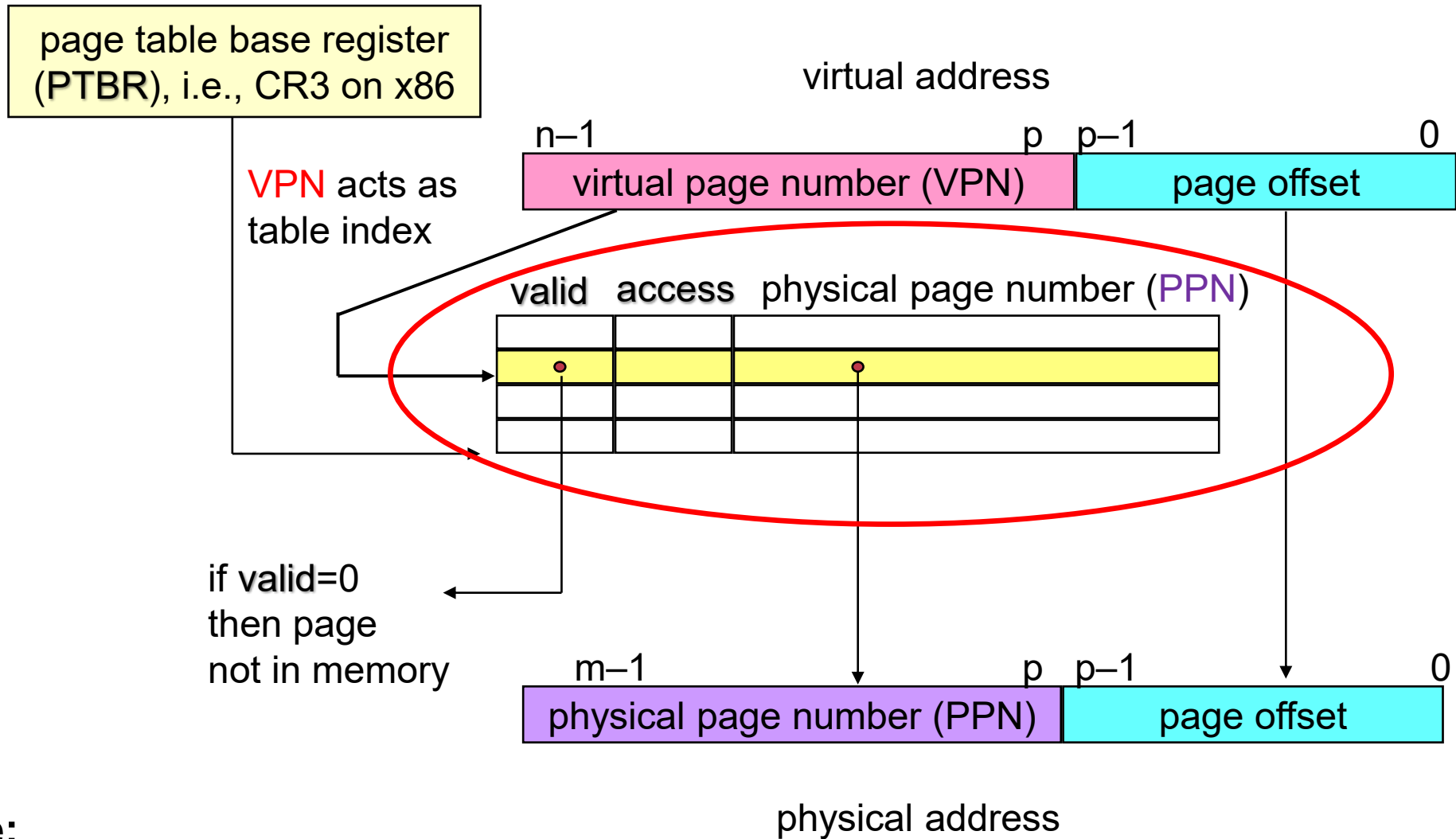


Notice that the page offset bits don't change as a result of translation

Address Translation

- **Basic Parameters**
 - **N** = 2^n = Virtual address limit
 - **M** = 2^m = Physical address limit
 - **P** = 2^p = page size (bytes)
- **Components of the virtual address (VA)**
 - **VPO**: Virtual page offset
 - **VPN**: Virtual page number
- **Components of the physical address (PA)**
 - **PPO**: Physical page offset (same as VPO)
 - **PPN**: Physical page number

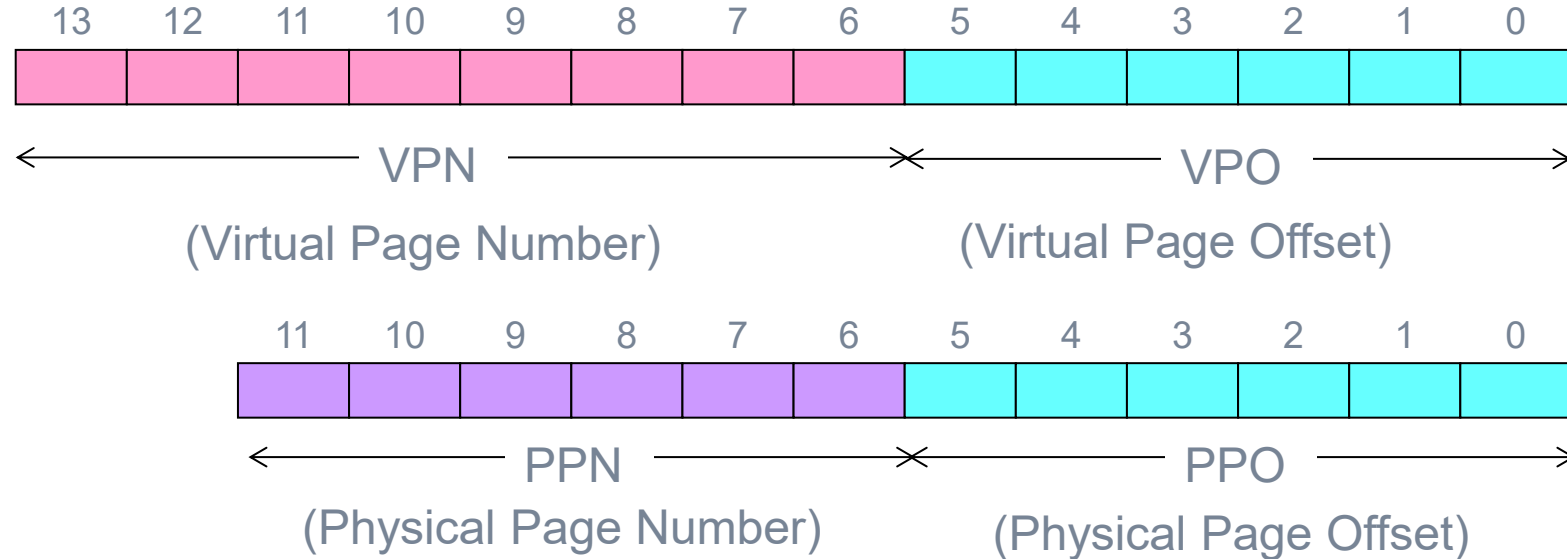
Address Translation via Page Table



Page table:
stored in physical memory managed by OS

Simple Memory System Example

- Addressing
 - 14-bit virtual addresses
 - 12-bit physical address
 - Page size = 64 bits (6-bit)



Simple Memory System Page Table

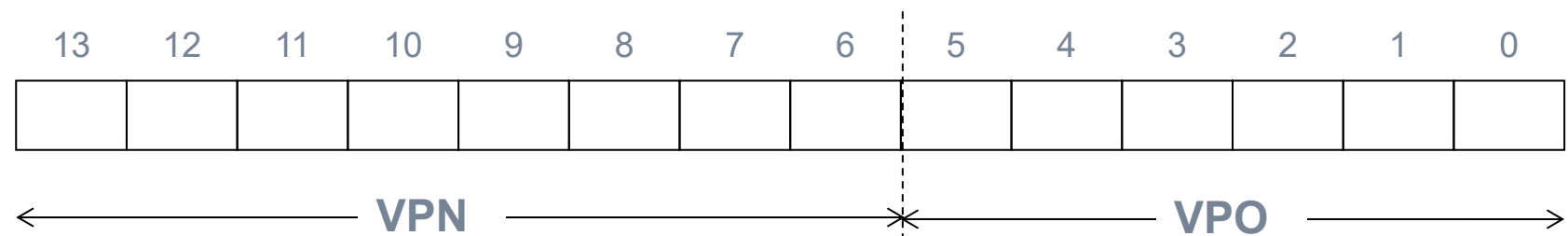
- Only show first 16 entries

	PPN	Valid
00	28	1
01	-	0
02	33	1
03	02	1
04	-	0
05	16	1
06	-	0
07	-	0

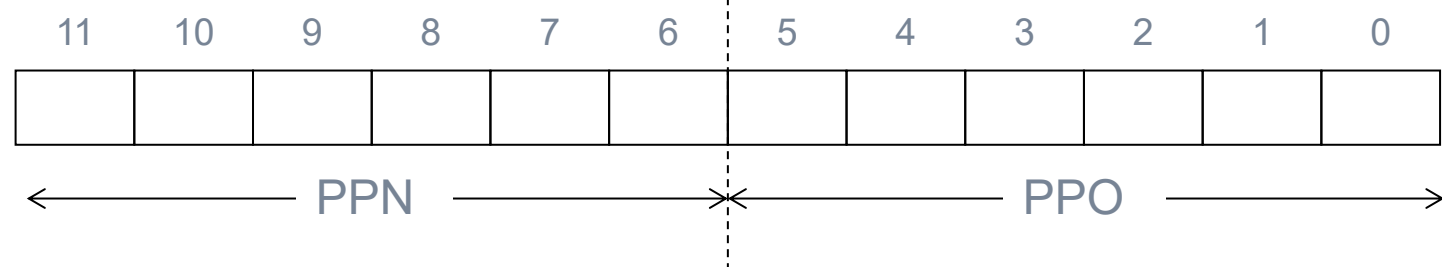
	PPN	Valid
08	13	1
09	17	1
0A	09	1
0B	-	0
0C	-	0
0D	2D	1
0E	11	1
0F	0D	1

Address Translation Example

Virtual Address: 0x03D4



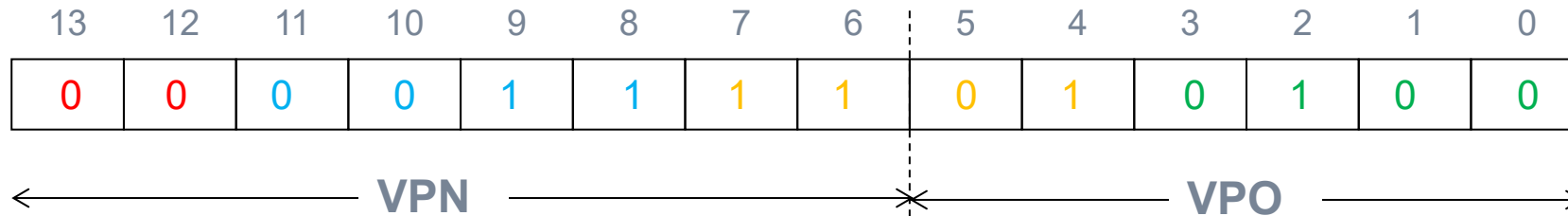
VPN: ____ VPO: ____ Page Fault? ____



PPN: ____ PPO: ____ PA: ____

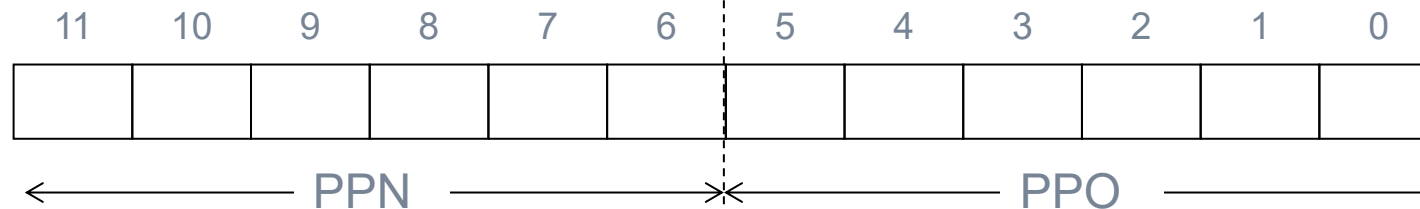
Address Translation Example

Virtual Address: 0x03D4



VPN: 0x0f VPO: 0x14

Page Fault? ____



PPN: ____ PPO: 0x14

PA: ____

Simple Memory System Page Table

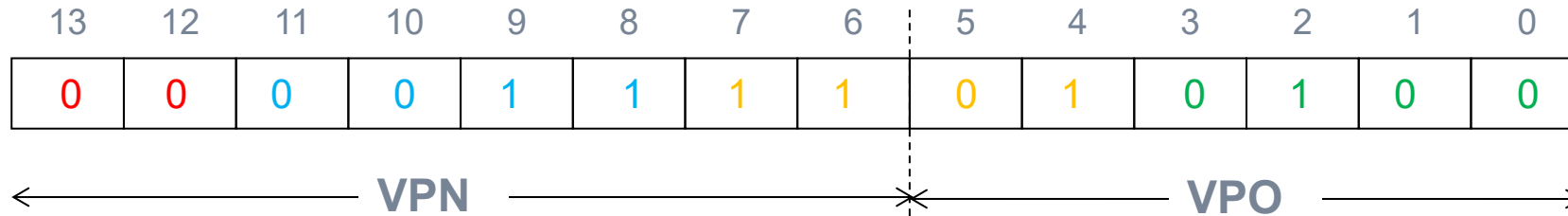
- Only show first 16 entries

	PPN	Valid
00	28	1
01	-	0
02	33	1
03	02	1
04	-	0
05	16	1
06	-	0
07	-	0

	PPN	Valid
08	13	1
09	17	1
0A	09	1
0B	-	0
0C	-	0
0D	2D	1
0E	11	1
0F	0D	1

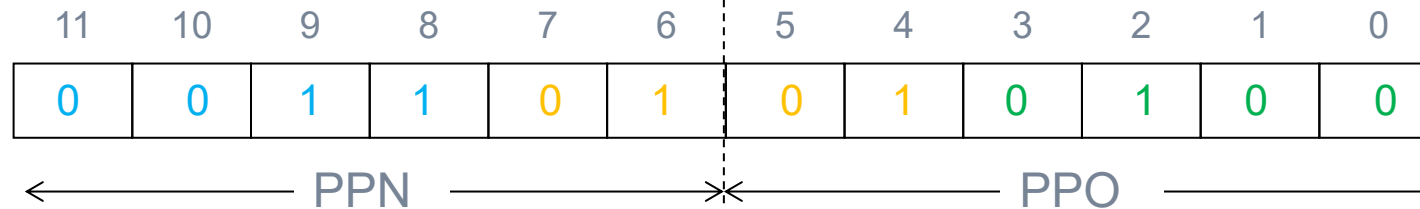
Address Translation Example

Virtual Address: 0x03D4



VPN: 0x0f VPO: 0x14

Page Fault? No

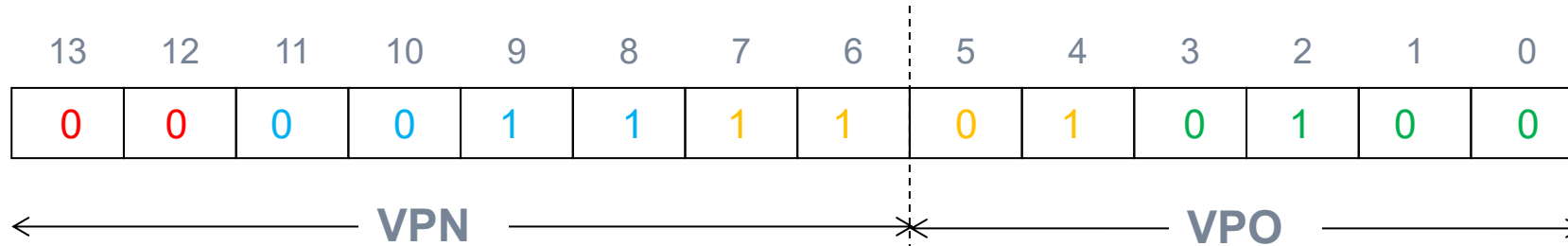


PPN: 0x0D VPO: 0x14

PA: 0x

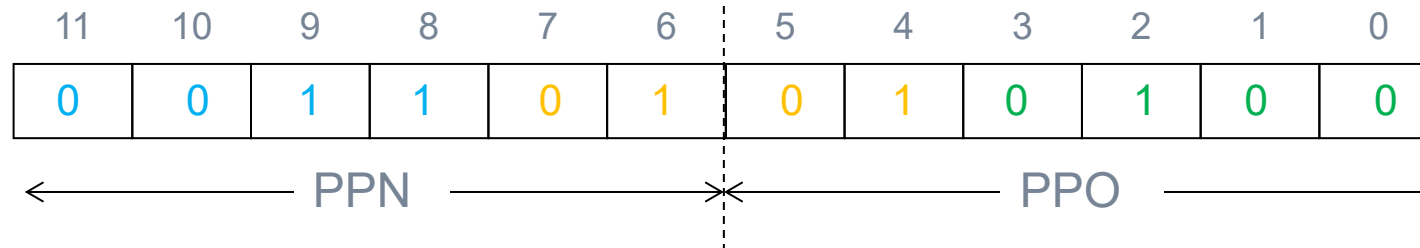
Address Translation Example

Virtual Address: 0x03D4



VPN: 0x0f VPO: 0x14

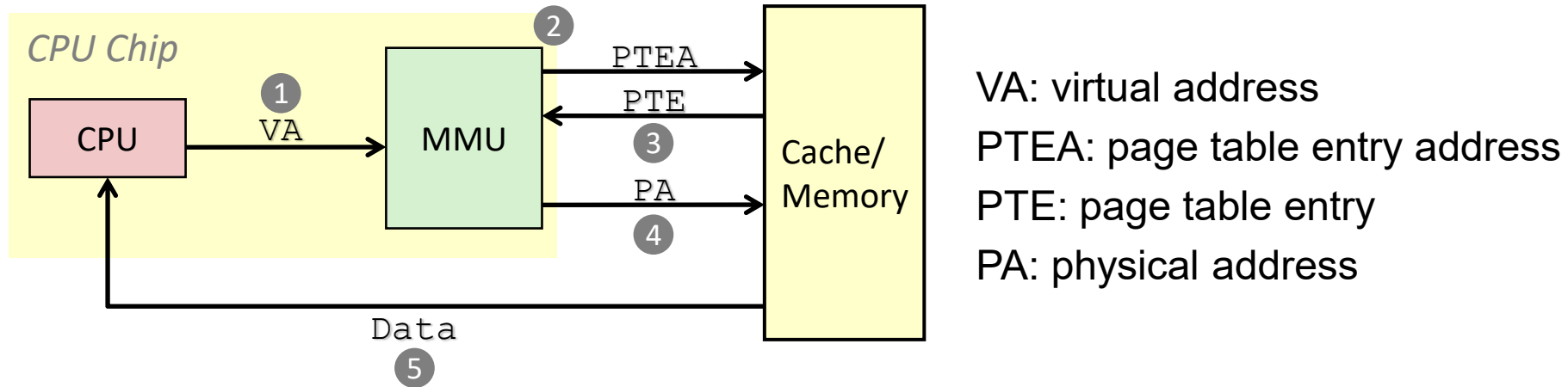
Page Fault? No



PPN: 0x0D VPO: 0x14

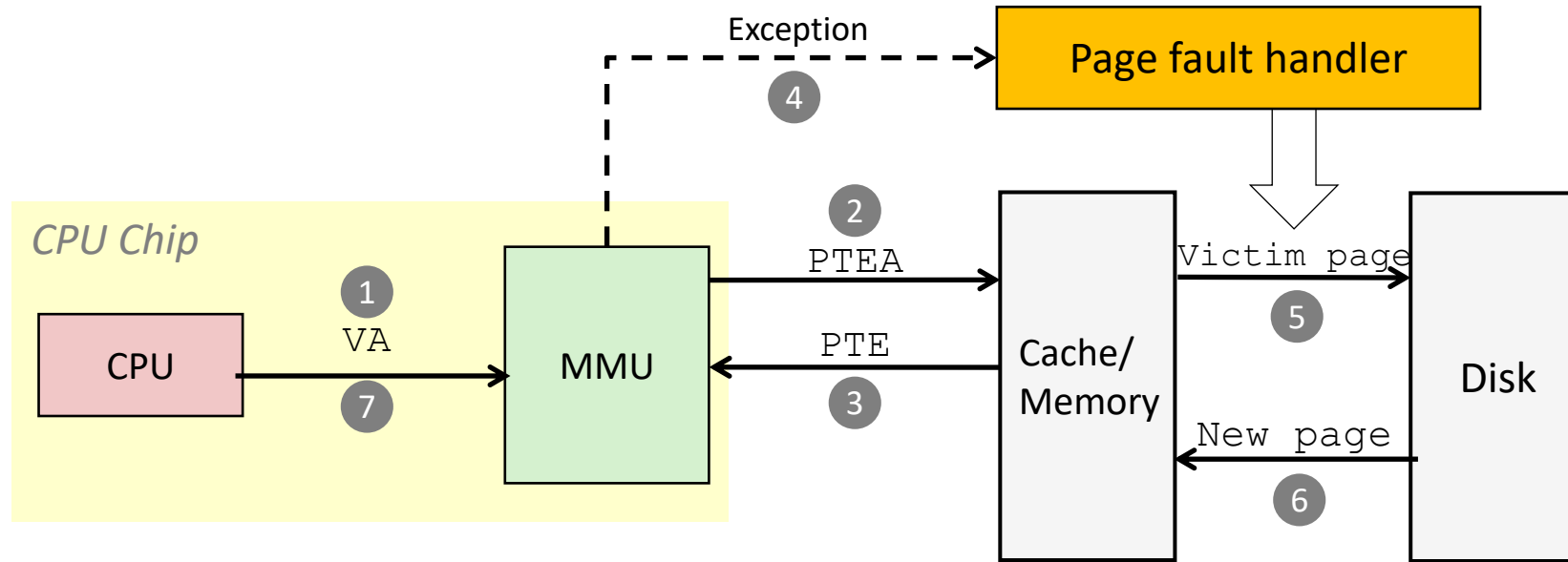
PA: 0x354

Page Hit



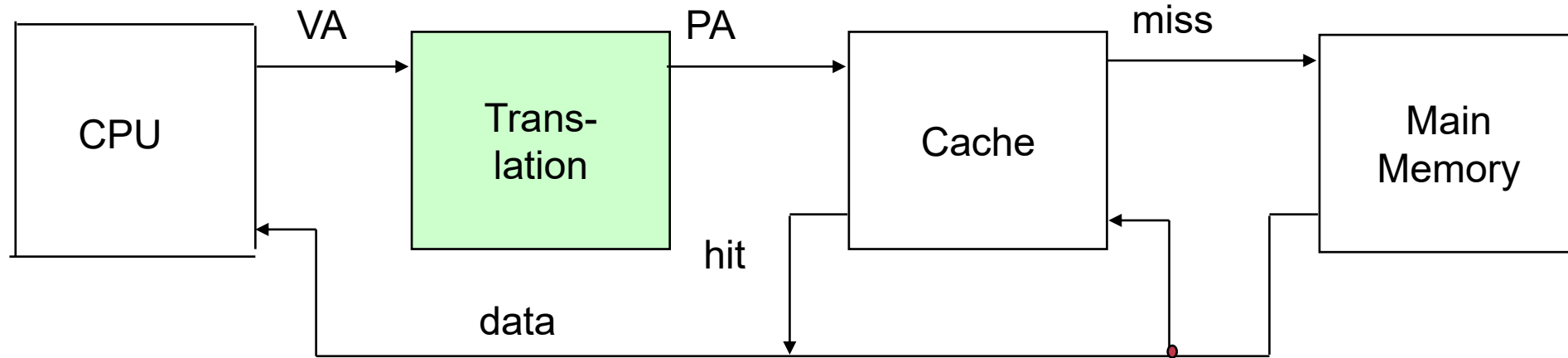
- 1) Processor sends virtual address to MMU
- 2-3) MMU fetches PTE from page table in memory
- 4) MMU sends physical address to cache/memory
- 5) Cache/memory sends data word to processor

Page Faults



- 1) Processor sends virtual address to MMU
- 2-3) MMU fetches PTE from page table in memory
- 4) Valid bit is zero, so MMU triggers page fault exception
- 5) Handler identifies victim (and, if dirty, pages it out to disk) (optional)
- 6) Handler pages in new page and updates PTE in memory
- 7) Handler returns to original process, restarting faulting instruction

Integrating Caches and VM



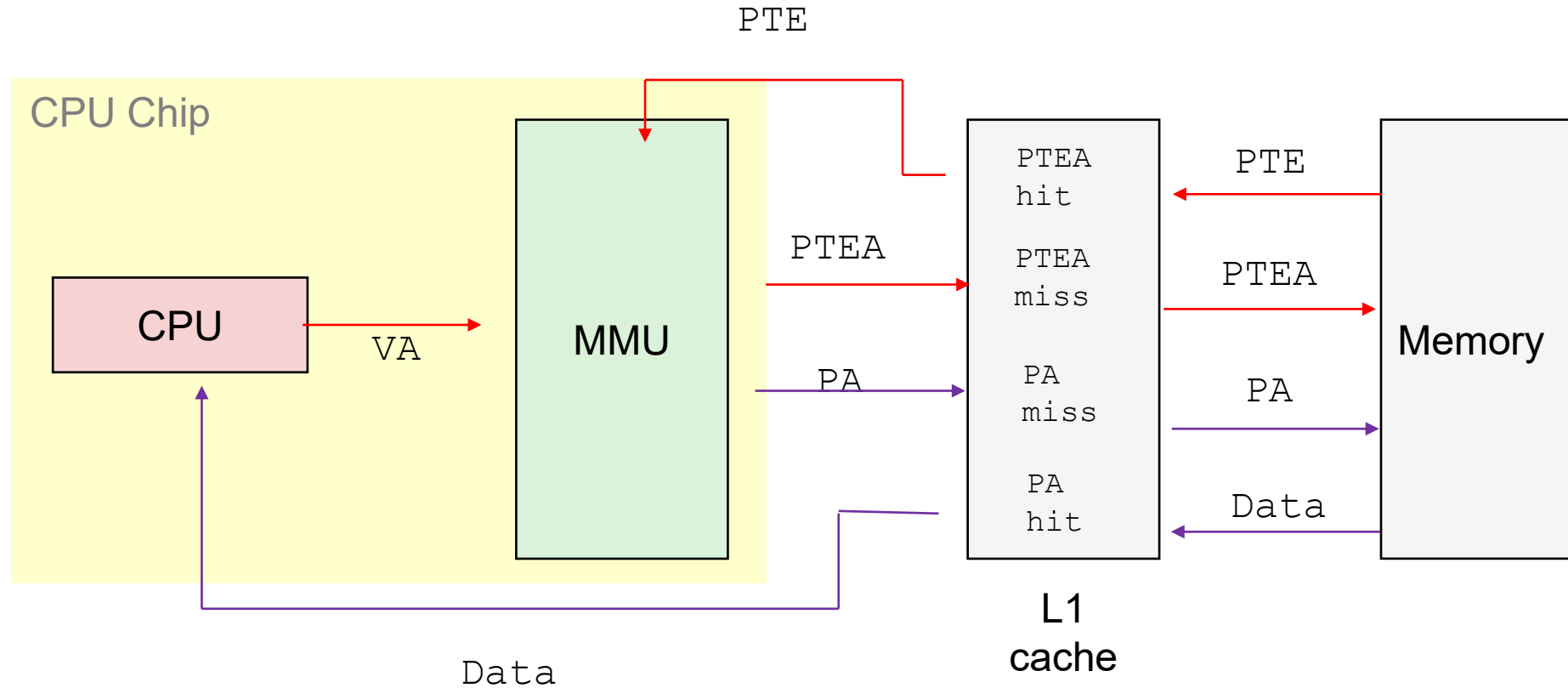
Integrating Caches and VM

- **Most Caches “Physically Addressed”**
 - Accessed by physical addresses
 - Allows multiple processes to have blocks in cache at same time
 - Allows multiple processes to share pages
 - Cache doesn’t need to be concerned with protection issues
 - Access rights checked as part of address translation

Integrating Caches and VM

- **Perform Address Translation Before Cache Lookup**
 - But this could involve a memory access itself (of the PTE)
 - Of course, page table entries can also become cached

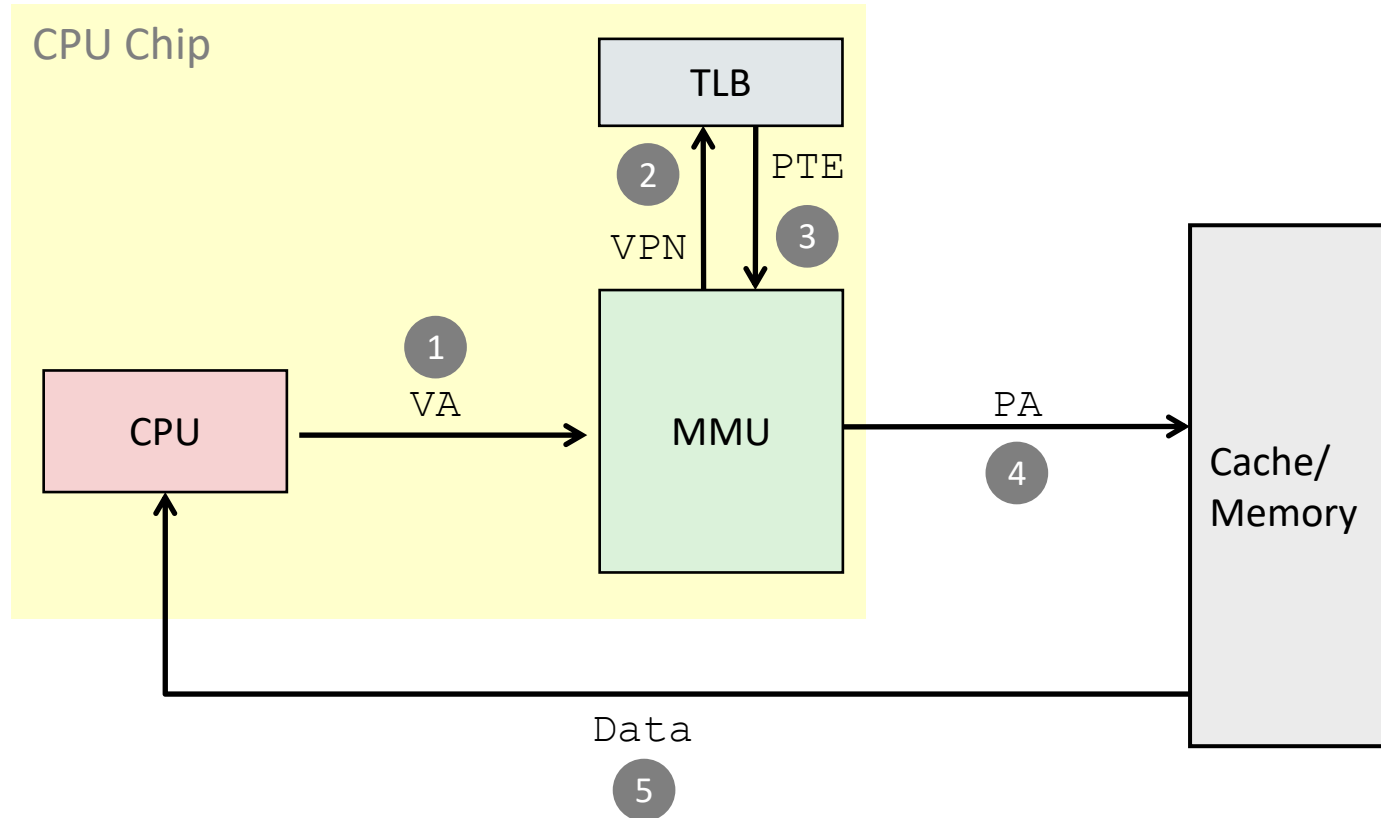
Integrating Caches and VM



Speeding up Translation with a TLB

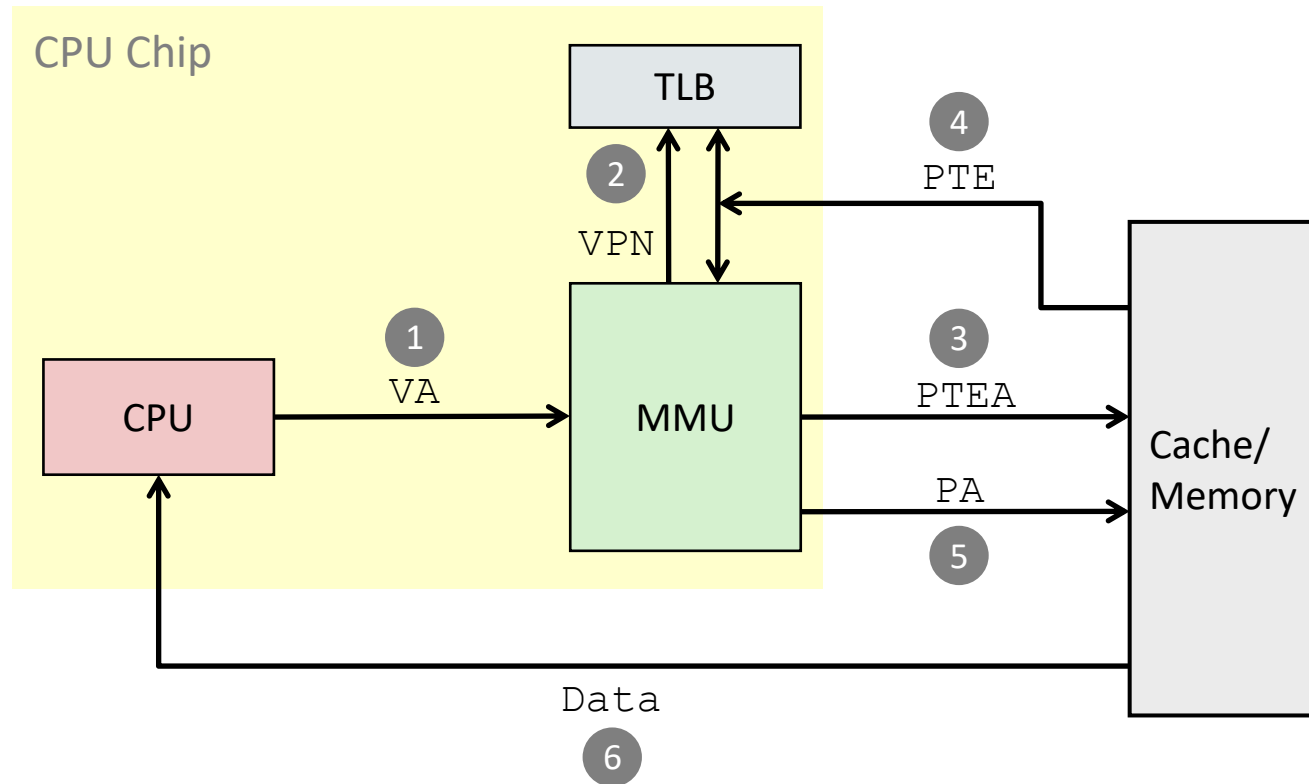
- **Translation Lookaside Buffer (TLB)**
 - Small hardware cache in MMU
 - Maps virtual page numbers to physical page numbers

TLB Hit



A TLB hit eliminates a memory access

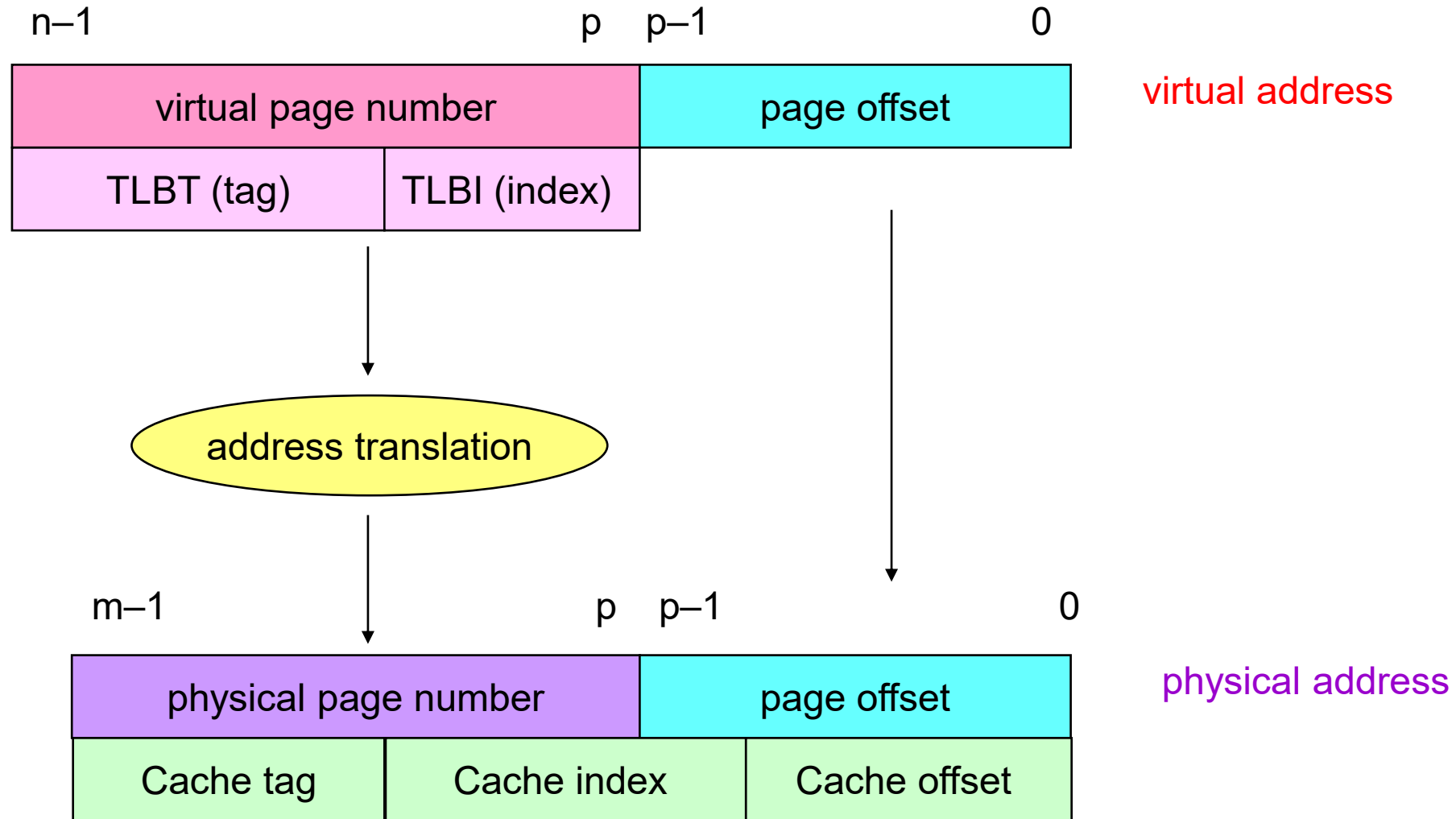
TLB Miss



A TLB miss incurs an additional memory access (PTE)

Q: Fortunately, TLB misses are usually not frequently. Why?

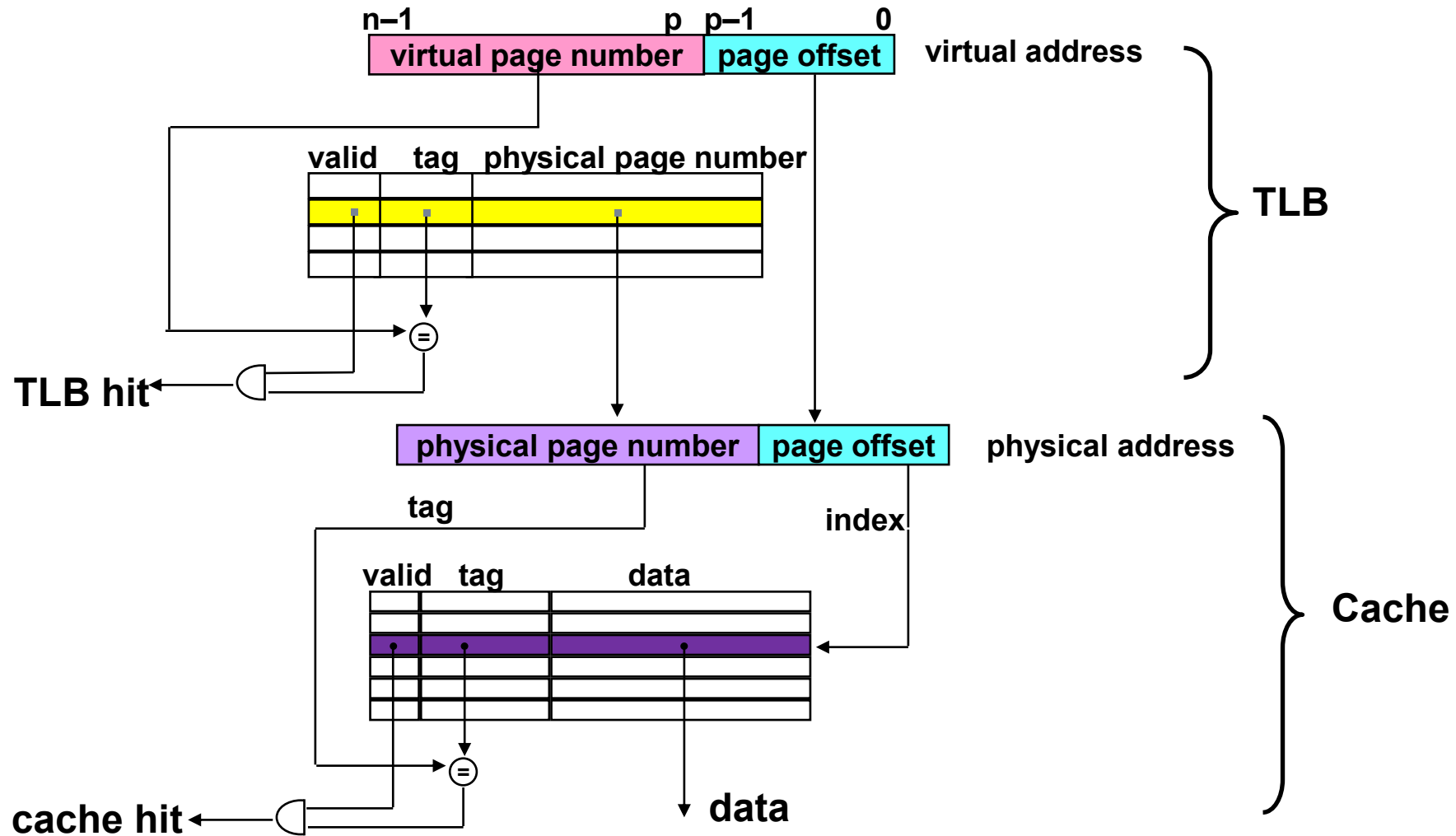
Speeding up Translation with a TLB



Address Translation

- **Components of the virtual address (VA)**
 - **VPO**: Virtual page offset
 - **VPN**: Virtual page number
 - **TLBI**: TLB index
 - **TLBT**: TLB tag
- **Components of the physical address (PA)**
 - **PPO**: Physical page offset (same as VPO)
 - **PPN**: Physical page number
 - **CO**: Byte offset within cache line
 - **CI**: Cache index
 - **CT**: Cache tag

Speeding up Translation with a TLB



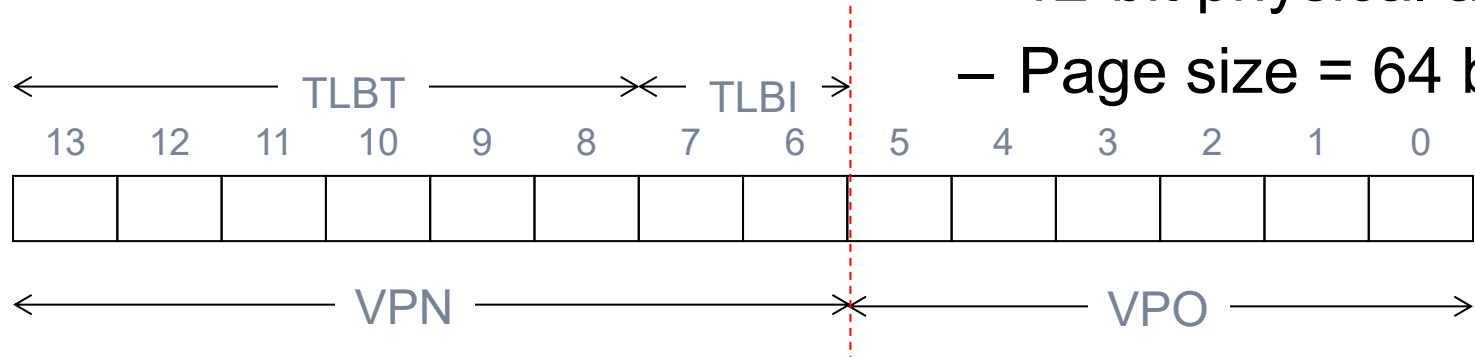
Simple Memory System TLB

- **TLB**

- 16 entries
- 4-way associative (2-bit)

- **Addressing**

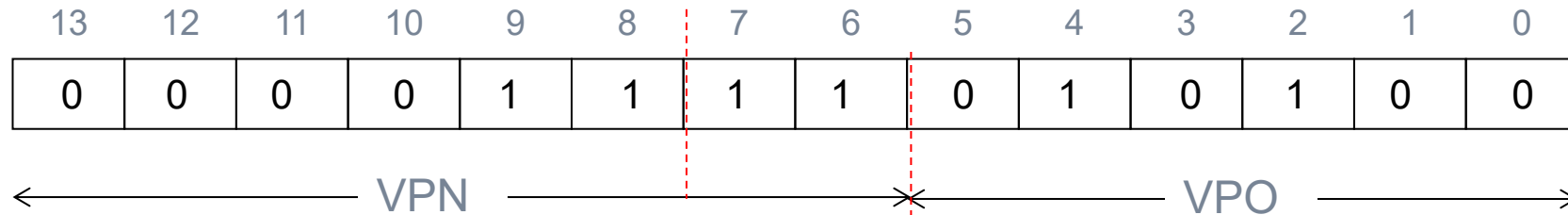
- 14-bit virtual addresses
- 12-bit physical address
- Page size = 64 bits (6-bit)



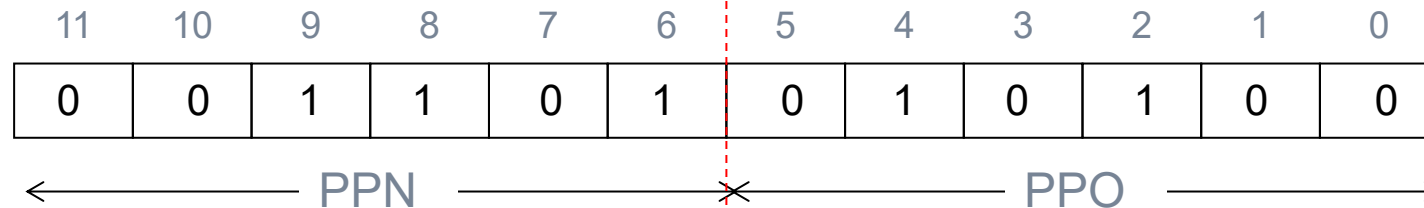
	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	03	-	0	09	0D	1	00	-	0	07	02	1
1	03	2D	1	02	-	0	04	-	0	0A	-	0
2	02	-	0	08	-	0	06	-	0	03	-	0
3	07	-	0	03	0D	1	0A	34	1	02	-	0

Address Translation Example

Virtual Address 0x03D4



VPN: 0x0f TLBI: 0x03 TLBT: 0x03 TLB Hit? __ Page Fault? __



PPN: ____ PPO: ____

PA: ____

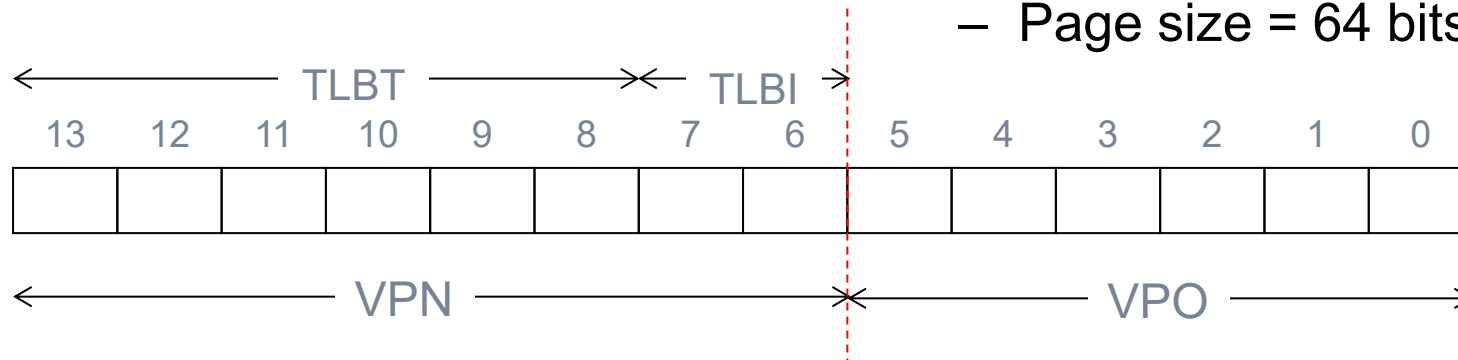
Simple Memory System TLB

- **TLB**

- 16 entries
- 4-way associative (2-bit)

- **Addressing**

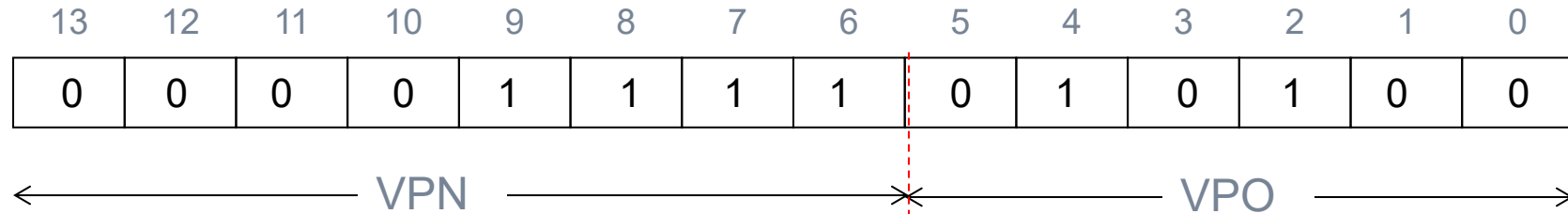
- 14-bit virtual addresses
- 12-bit physical address
- Page size = 64 bits (6-bit)



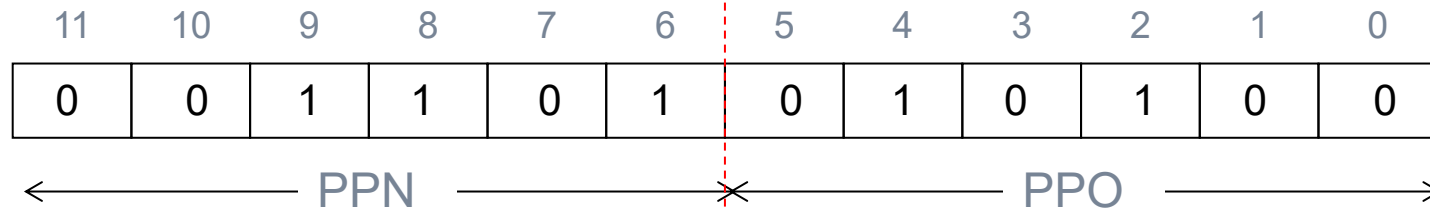
	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	03	-	0	09	0D	1	00	-	0	07	02	1
1	03	2D	1	02	-	0	04	-	0	0A	-	0
2	02	-	0	08	-	0	06	-	0	03	-	0
3	07	-	0	03	0D	1	0A	34	1	02	-	0

Address Translation Example

Virtual Address 0x03D4



VPN: 0x0f TLBI: 0x03 TLBT: 0x03 TLB Hit? Yes Page Fault? No



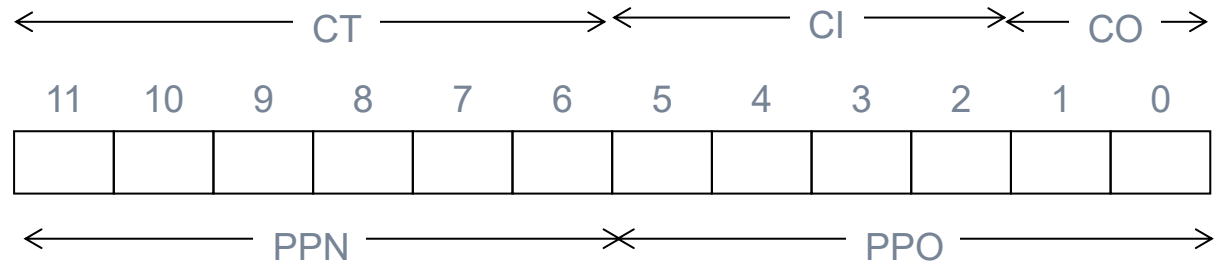
PPN: 0x0D PPO: 0x14

PA: 0x354

Simple Memory System Cache

- Cache

- 16 lines
- 4-byte line size
- Direct mapped

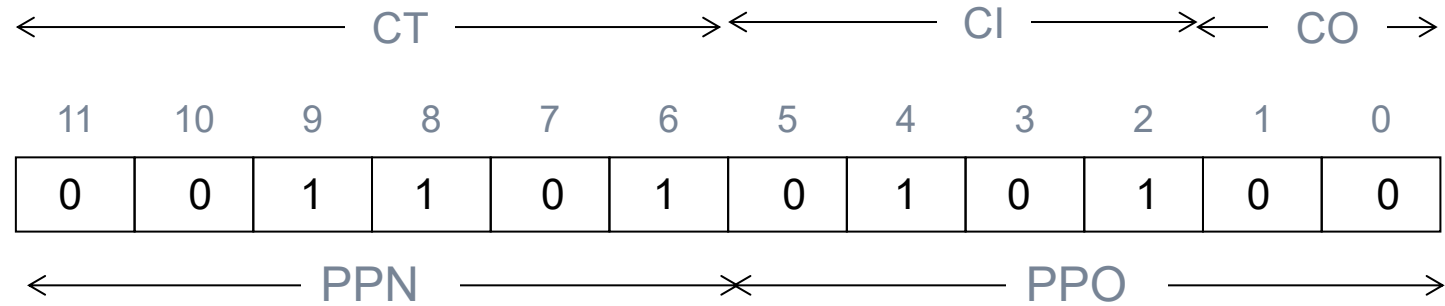


Idx	Tag	Valid	B0	B1	B2	B3
0	19	1	99	11	23	11
1	15	0	-	-	-	-
2	1B	1	00	02	04	08
3	36	0	-	-	-	-
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	-	-	-	-
7	16	1	11	C2	DF	03

Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	-	-	-	-
A	2D	1	93	15	DA	3B
B	0B	0	-	-	-	-
C	12	0	-	-	-	-
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	-	-	-	-

Address Translation Example

PA: 0x354



Offset: 0x0 CI: 0x05 CT: 0x0D Hit? __ Byte:

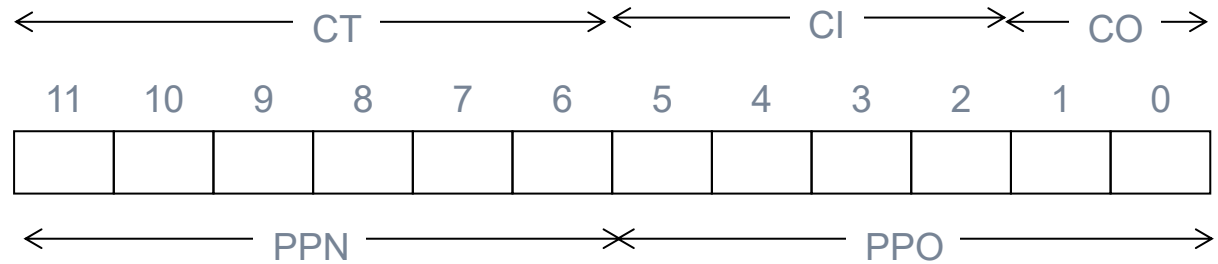
Simple Memory System Cache

- Cache

- 16 lines

- 4-byte line size

- Direct mapped

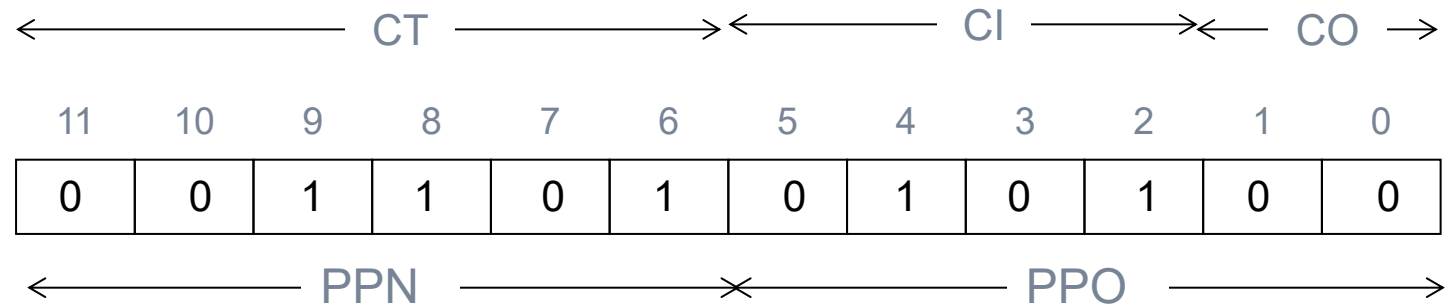


	Tag	Valid	B0	B1	B2	B3
0	19	1	99	11	23	11
1	15	0	-	-	-	-
2	1B	1	00	02	04	08
3	36	0	-	-	-	-
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	-	-	-	-
7	16	1	11	C2	DF	03

	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	-	-	-	-
A	2D	1	93	15	DA	3B
B	0B	0	-	-	-	-
C	12	0	-	-	-	-
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	-	-	-	-

Address Translation Example

PA: 0x354



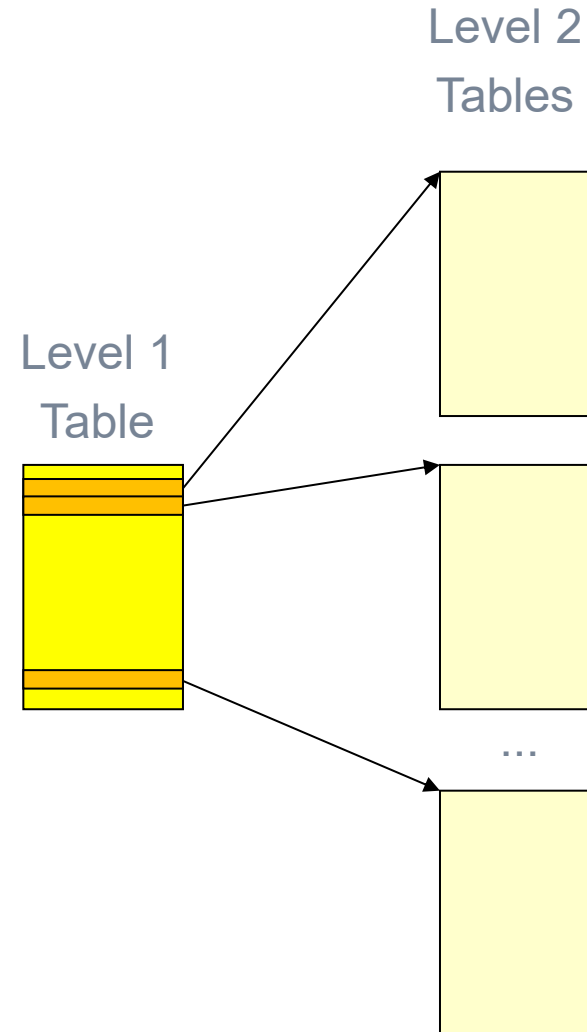
Offset: 0x0 CI: 0x05 CT: 0x0D Hit? Yes Byte: 0x36

Multi-Level Page Tables

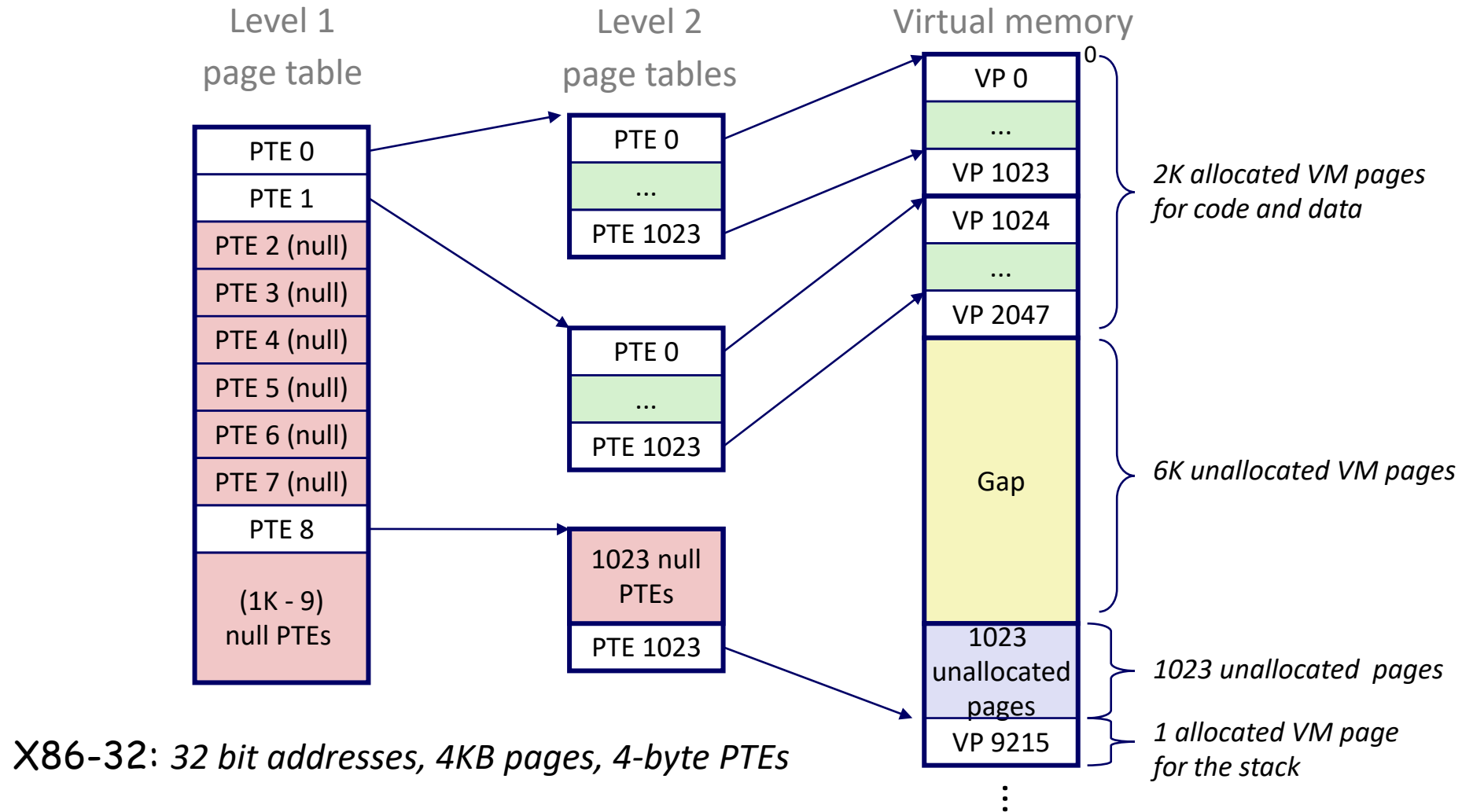
- **Given:**
 - X86: 32-bit address space
4KB (2^{12}) page size, 4-byte PTE
 - X86-64: 48-bit address space
4KB (2^{12}) page size, 8-byte PTE
- **Problem:**
 - X86: Would need a **4 MB** page table!
 - $2^{20} * 4$ bytes (20bit = 32bit – 12bit)
 - X86-64: Would need a **512 GB** page table!
 - $2^{36} * 8$ bytes (36bit = 48bit – 12bit)

Multi-Level Page Tables

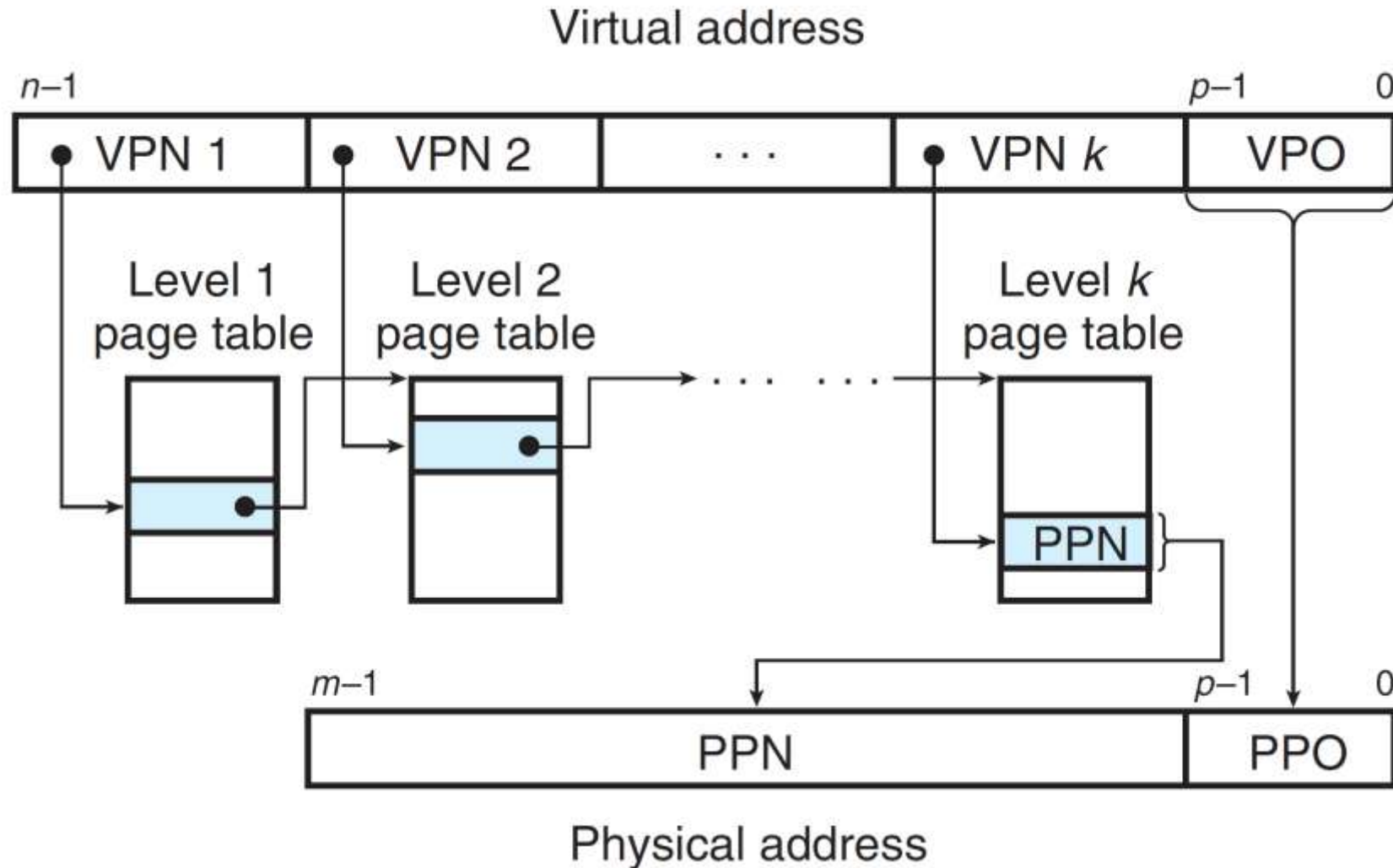
- **Common solution**
 - multi-level page tables
 - e.g., 2-level page table for X86-32
 - **Level 1 table:** 1024 entries, each of which points to a Level 2 page table.
 - **Level 2 table:** 1024 entries, each of which points to a page



Multi-Level Page Tables



Multi-Level Page Tables



X86-64

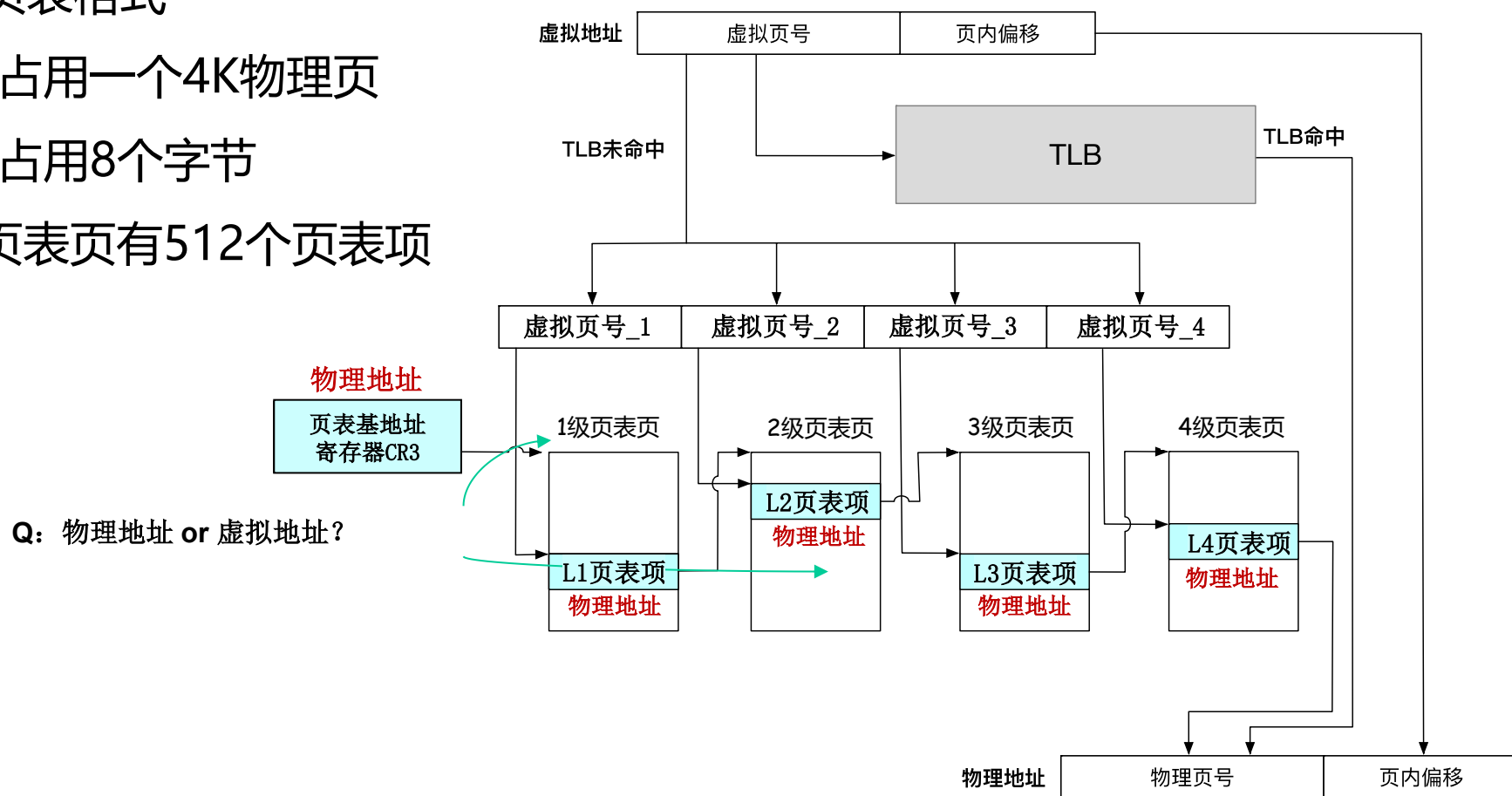
- **canonical address**
 - 48 effective bits in one 64-bit virtual address
 - The upper 16 bits need to be copies of the 48th bit

Start addr	Offset	End addr	Size	VM area description
0000_0000_0000_0000	0	0000_7fff_ffff_ffff	128 TiB	user-space virtual memory
0000_8000_0000_0000	+128 TiB	ffff_7fff_ffff_ffff	~16M TiB	non-canonical
ffff_8000_0000_0000	-128 TiB	ffff_ffff_ffff_ffff	128 TiB	kernel-space virtual memory

4-Level Page Table

硬件规定的页表格式

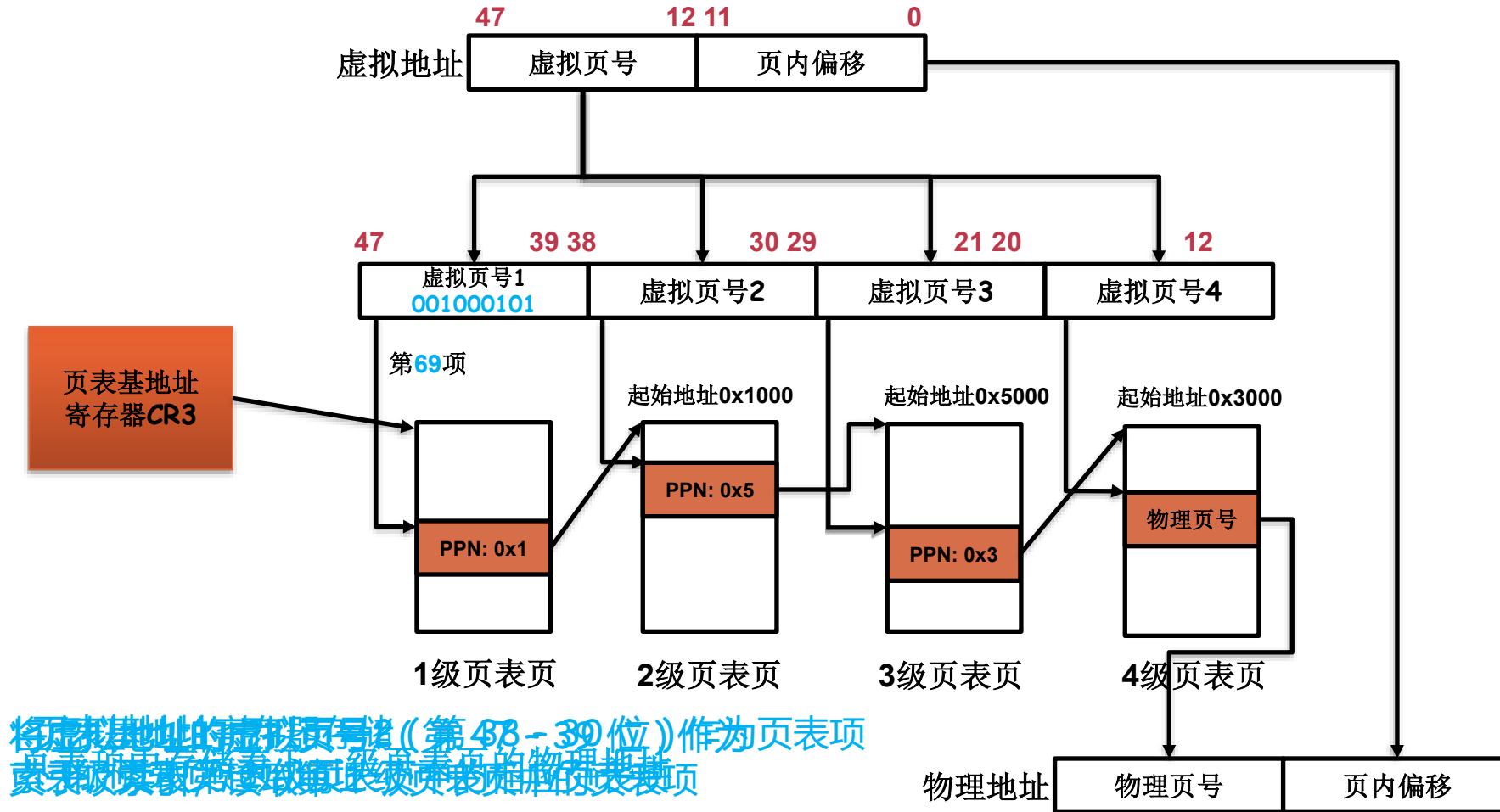
- 每个页表页占用一个4K物理页
- 每个页表项占用8个字节
 - 每个页表页有512个页表项



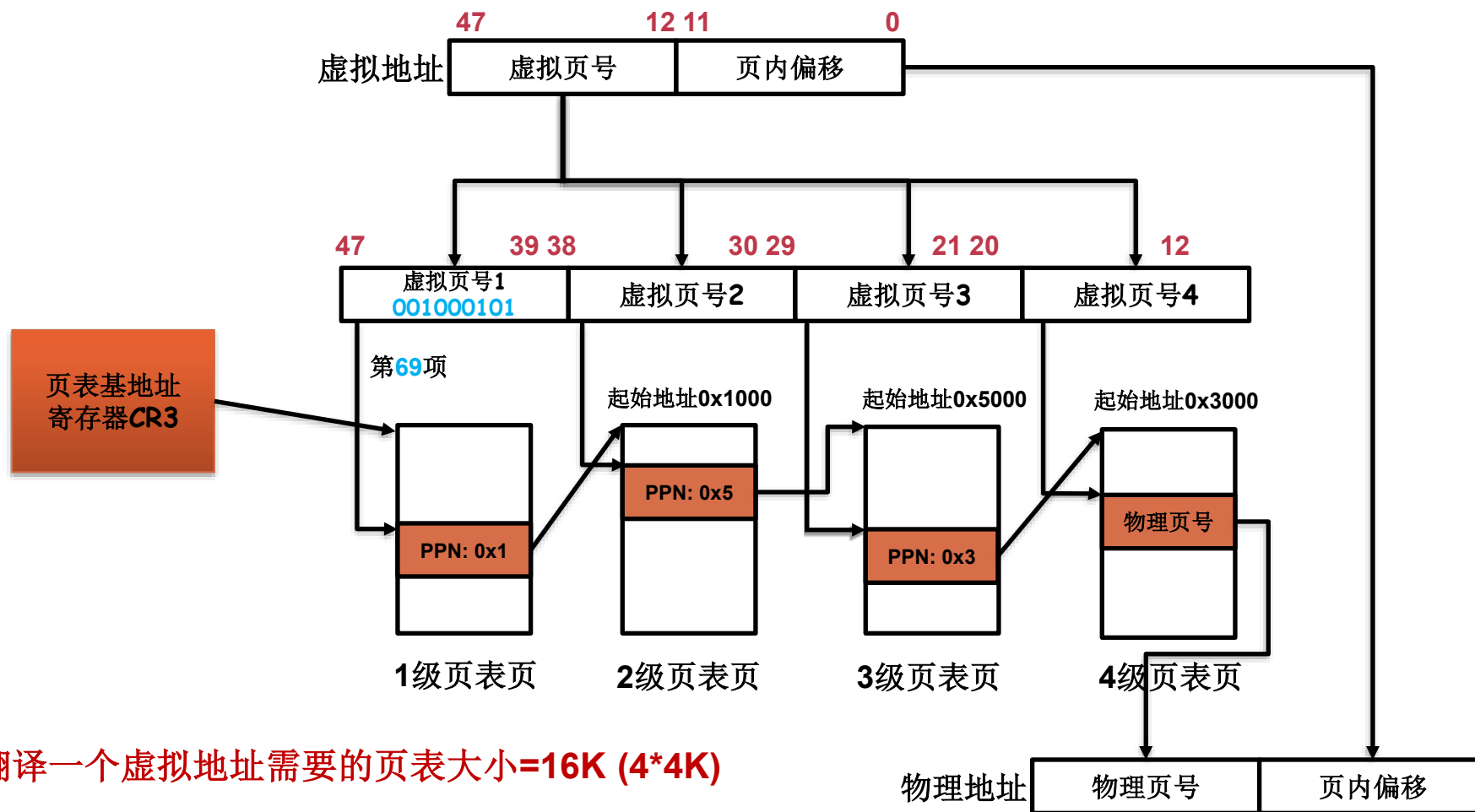
64-Bit Address

- 「63: 48」 16-bit
 - 必须全是0或者全是1（软件用法：一般应用程序地址是0，内核地址是1）
 - 和第47位相同
- 「47: 39」 9-bit: 1级页表索引
- 「38: 30」 9-bit: 2级页表索引
- 「29: 21」 9-bit: 3级页表索引
- 「20: 12」 9-bit: 4级页表索引
- 「11: 0」 12-bit: 页内偏移

Example



Example



翻译一个虚拟地址需要的页表大小=16K (4*4K)

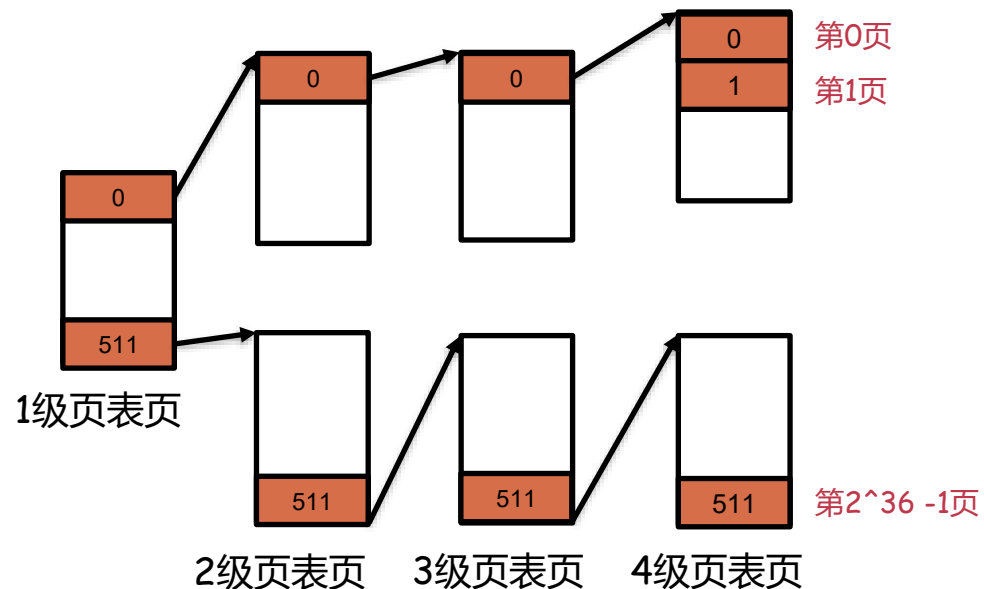
尽管其余511项没有用，但是单个页表页一旦分配就是占用4K

问：若页表共占用16K，能够翻译的虚拟地址范围是？

Example

假设进程只用到0, 1, $2^{36} - 1$ 三个虚拟页

- 单级页表, 共需 2^{36} 个页表项, 共占用 2^{39} B
- 4级页表, 仅需要一个0级页表, 2个一级页表, 2个二级页表, 2个三级页表, 共七个页表, 每个页表页占用(4KB), 共占用28KB

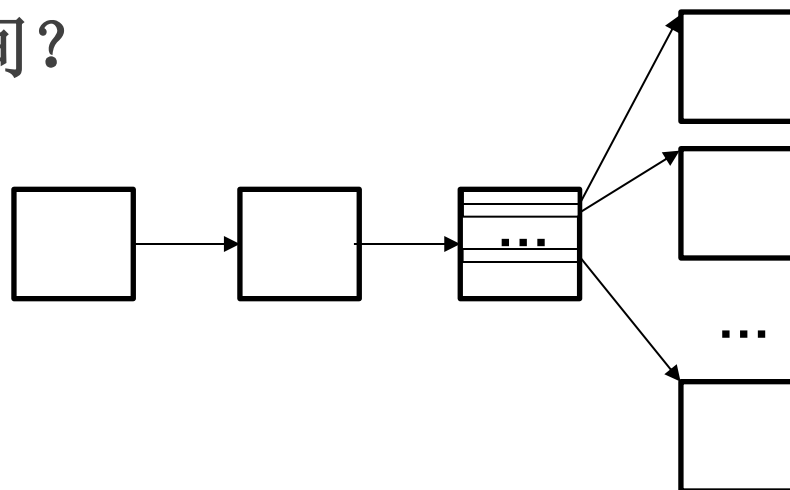


进程1的页表

0
1
...
...
...
...
...
...
$2^{36}-1$

Question

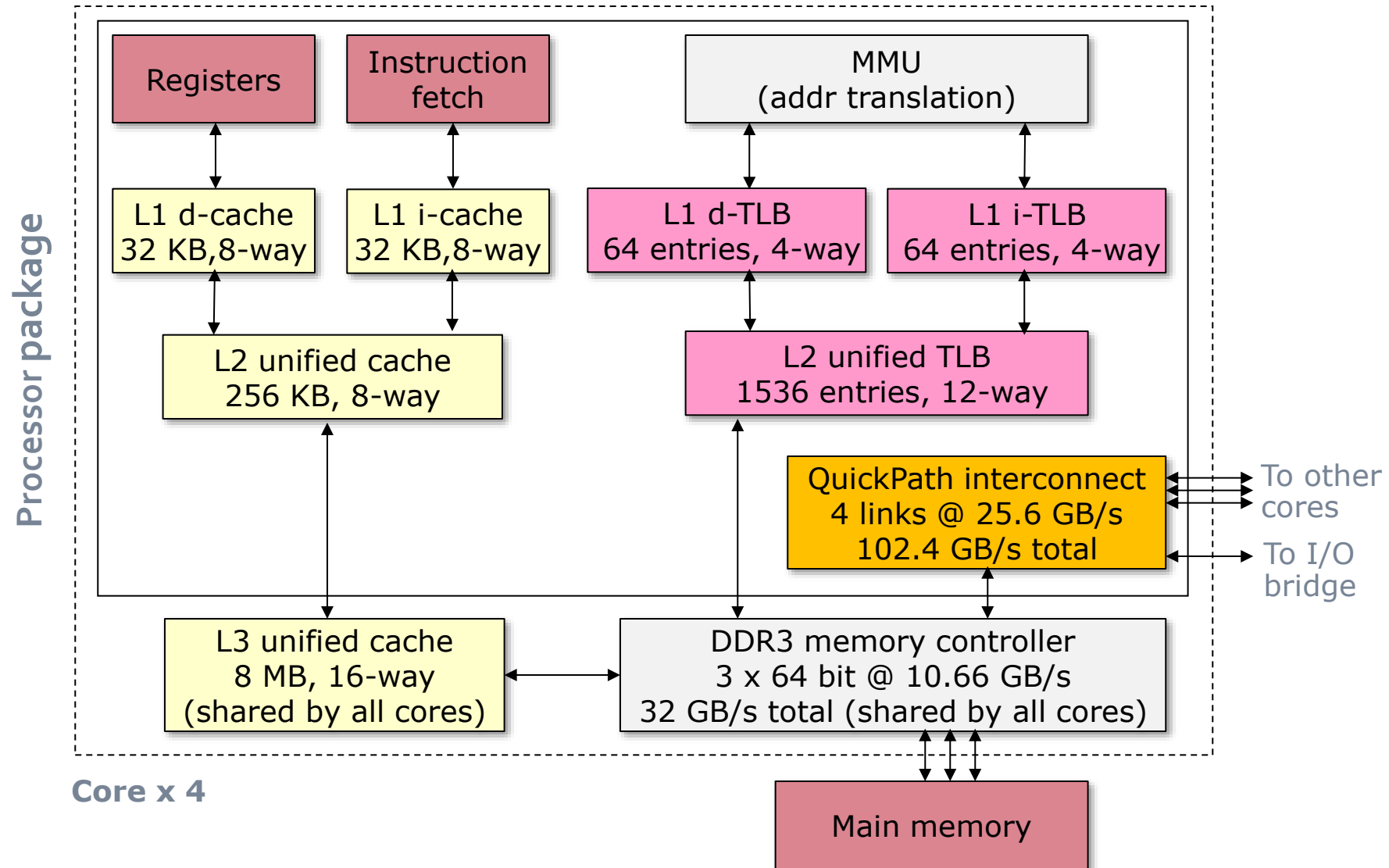
1. 多级页表是否一定比单级页表占用内存少？
2. 虚拟地址**0xFFF8080604FFF**对应的
各级页表索引？ **L4页表项索引： $(VA \& (0x1ff \ll 12)) \gg 12$**
3. 若在页表中填写虚拟地址 **0-16M** 的映射，则页表至少需要占用多少空间？



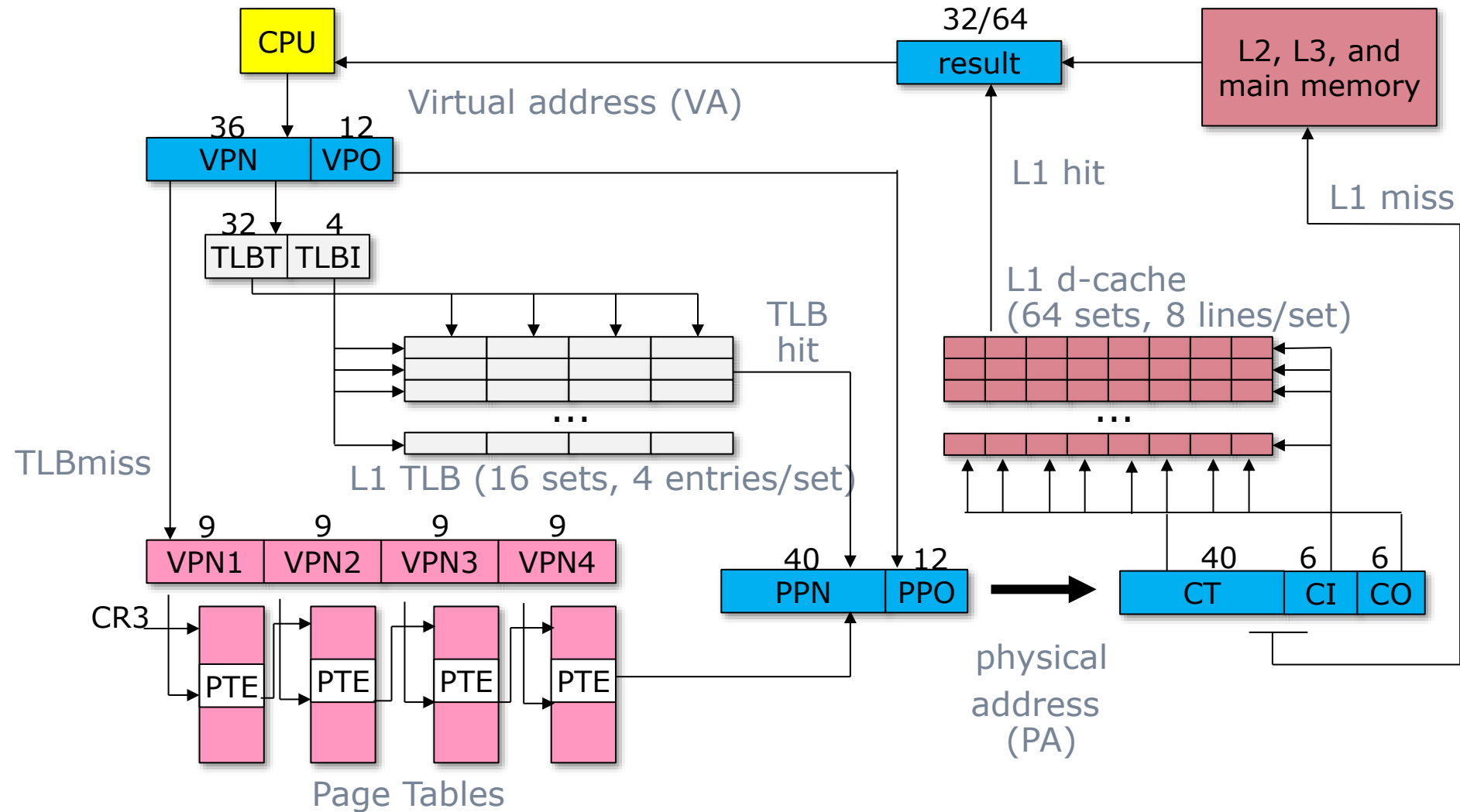
Core i7 Summery

- **Core i7**
 - The 64-bit Nehalem microarchitecture
 - Current support 48-bit (256 TB) virtual address space and 52-bit (4 PB) physical address space
 - Compatibility mode support 32-bit (4 GB) virtual and physical address spaces

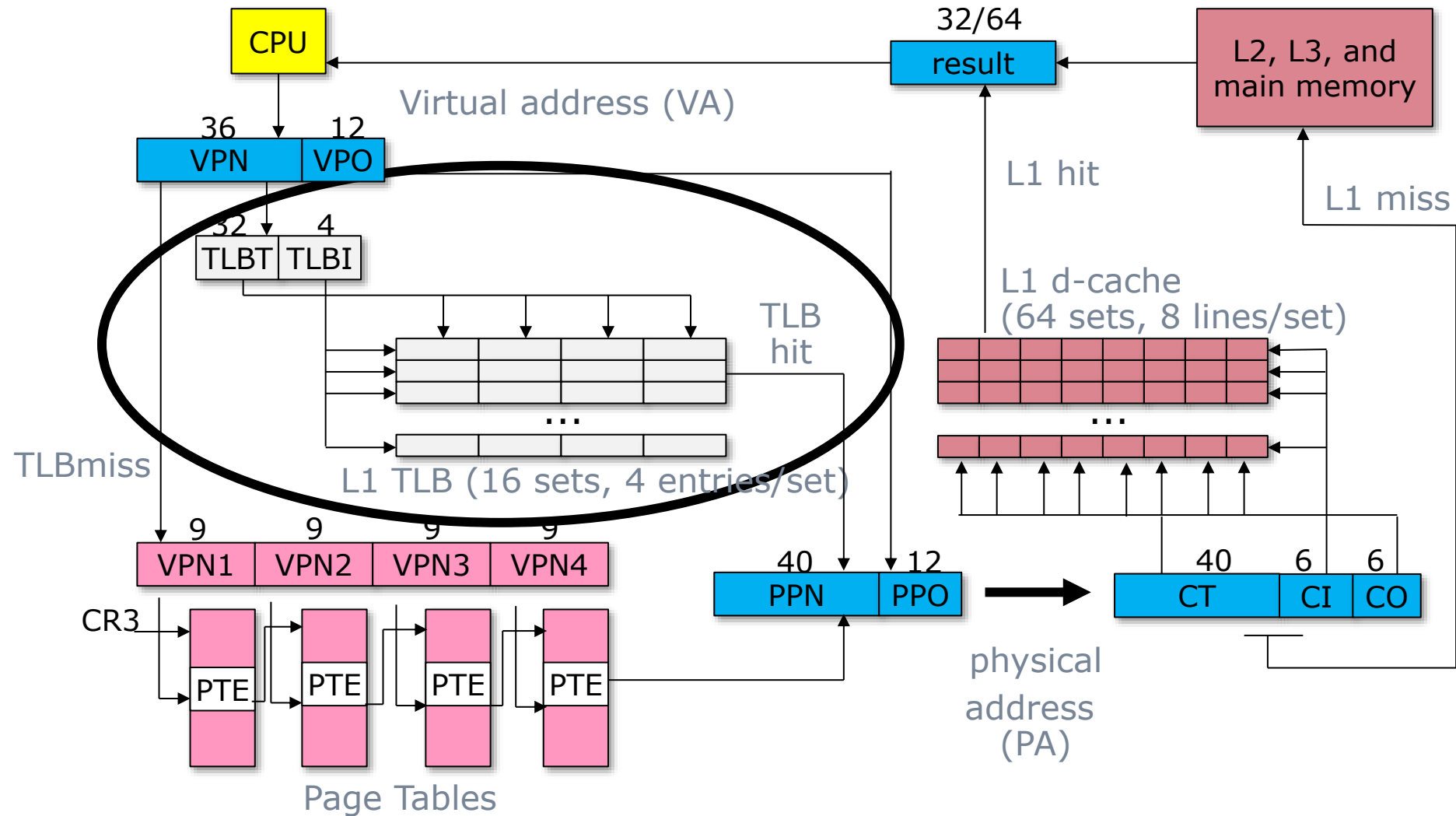
Core i7 Memory System (Skylake)



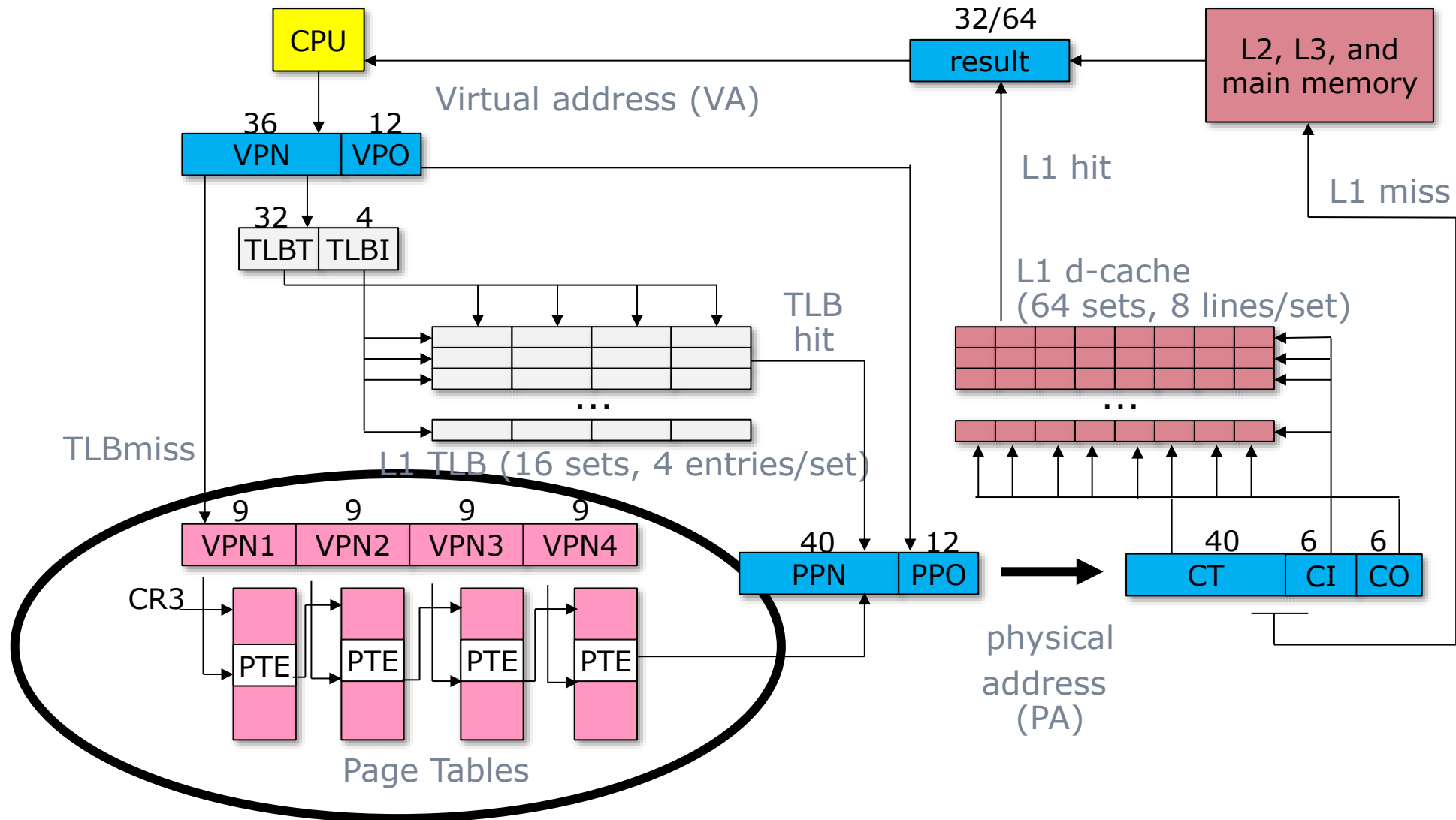
Core i7 Address Translation



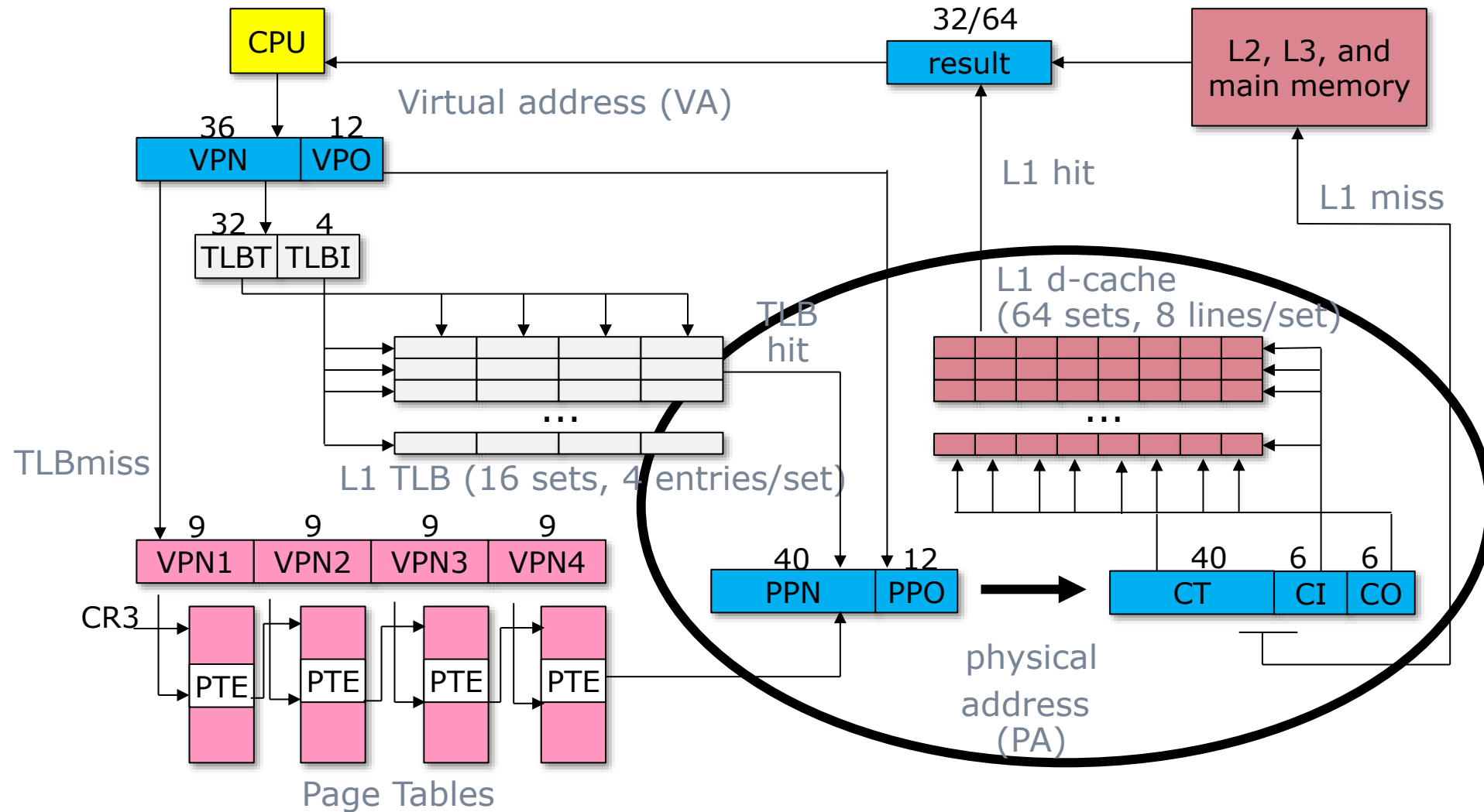
Core i7 Address Translation



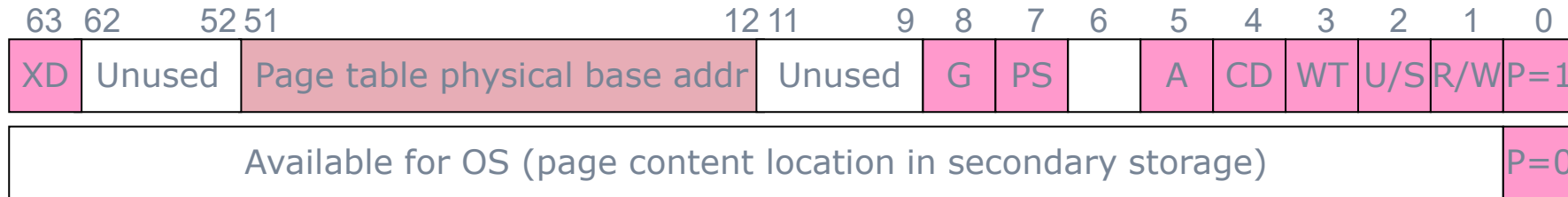
Core i7 Address Translation



Core i7 Address Translation



Level 1, Level 2 and Level 3 Page Table Entry



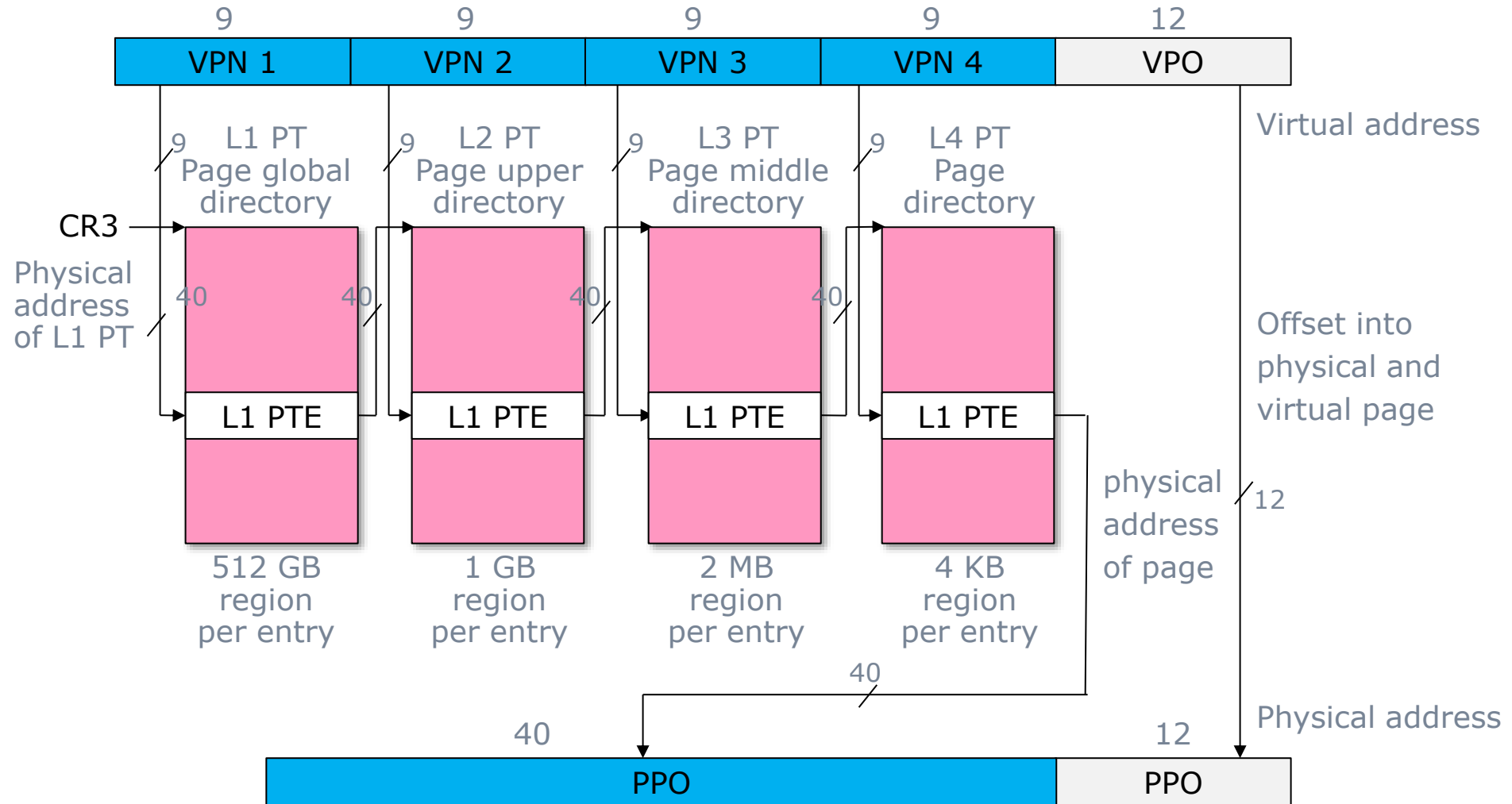
XD	Disable or enable instruction fetches
Base addr	40 most significant bits of base address of child page table (forces page tables to be 4KB aligned)
G	global page (don't evict from TLB on task switch)
PS	Page size either 4K or 2M or 1G(may set to 1 only for Level 2 or level 3 PTEs)
A	Reference bit (set by MMU on reads and writes, cleared by software)
CD	Cache disabled(1) or enabled(0) for child page table
WT	Write-through or write-back cache policy
U/S	User or supervisor(kernel) mode access permission
R/W	Read-only or read-write access permission
P	Child page table present in memory(1) or not(0)

Level 4 Page Table Entry

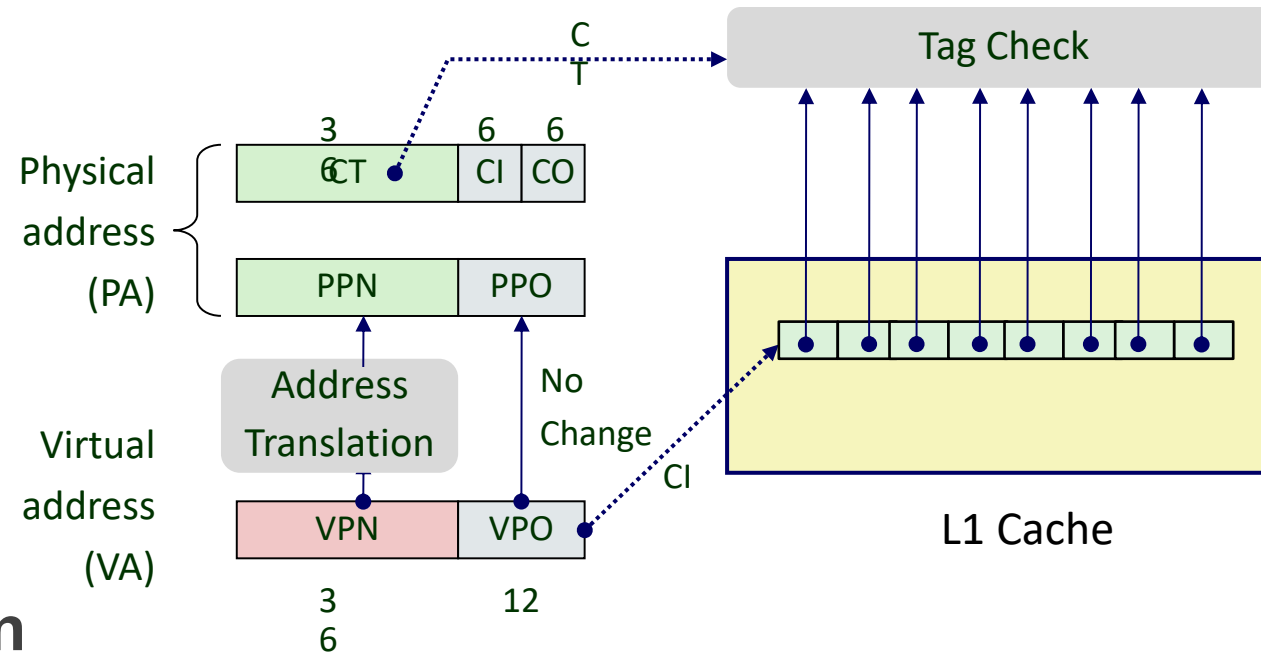
63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
XD	Unused	Page table physical base addr				Unused	G		D	A	CD	WT	U/S	R/W	P=1
Available for OS (page content location in secondary storage)															P=0

XD	Disable or enable instruction fetches
Base addr	40 most significant bits of base address of child page table (forces page tables to be 4KB aligned)
G	global page (don't evict from TLB on task switch)
D	Dirty bit (Set by MMU on writes, cleared by software)
A	Reference bit (set by MMU on reads and writes, cleared by software)
CD	Cache disabled(1) or enabled(0) for child page table
WT	Write-through or write-back cache policy
U/S	User or supervisor(kernel) mode access permission
R/W	Read-only or read-write access permission
P	Child page table present in memory(1) or not(0)

Page tables Translation



Cute Trick for Speeding Up L1 Access (VIPT)



- **Observation**

- Bits that determine CI identical in virtual and physical address
- Can index into cache while address translation taking place
- Generally we hit in TLB, so PPN bits (CT bits) available next
- “Virtually indexed, physically tagged”
- Cache carefully sized to make this possible

Summary

- **Programmer's view of virtual memory**
 - Each process has its own **private** linear address space
 - Cannot be corrupted by other processes
- **System view of virtual memory**
 - Uses memory efficiently by caching virtual memory pages
 - Efficient only because of locality
 - Simplifies memory management and programming
 - Simplifies protection by providing a convenient interpositioning point to check permissions